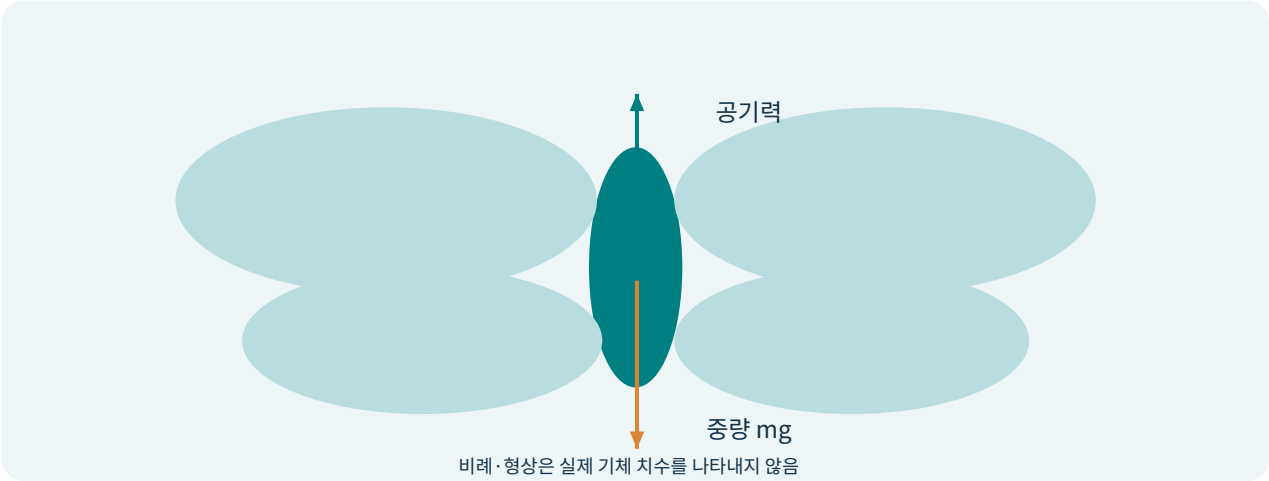
날아가는 나비를 만드는
RC 생체모방 나비 제작 전집

CAD · Blender · 3D 프린팅 · 기구 · 전기 · Arduino · 시험



비례·형상은 실제 기체 치수를 나타내지 않음

6권 구성 통합본 | 초보자용 절차 + 코드 원문 + 그림 해설

제작 학습용 초판 · 2026년 11월 11일

사용자 제공 S-DiY 사인파 코드와 배선도를 바탕으로 작성. 원본 도면 실물과 부품 사양 및 비행 결과가 없는 항목은 확정하지 않았습니다. 본문 그림은 별도 표기가 없으면 설명용 개념도이며 제작 치수 도면이 아닙니다.

실제 보드에서 컴파일·구동·비행 검증을 완료한 기체 매뉴얼이 아닙니다. 작업 순서와 확인 기준을 제공하는 제작 교재입니다.

읽기 전에 | 자료의 경계

이 책에 들어 있는 것

31개 장, 단계별 개념 그림, 사용자 제공 배선 사진, 원본 스케치 전체, 코드 구간별 주석 해설, 교육용 작은 진폭 시험 코드, 채널·사양·시험 기록 양식, 참고 문헌을 수록했다.

아직 없는 것

원본 DXF의 형상·치수, Blender 복구 파일 실물, 실제 출력 설정, 특정 부품 모델, 완성 기체의 질량·날개 치수·무게중심·비행 실측값은 없다. 이 항목은 빈칸 또는 확인 필요로 남겼다.

버전과 수치 읽는 법

원본 코드값은 90° 중심, 65° 진폭, 500~2500 µs 펄스 설정, D3/D9/D10/D6다. 작은 진폭 시험값과 치수 예시는 원본 설계값이 아니다. Blender 화면은 4.0 계열을 기준으로 설명하며 메뉴와 exporter는 설치 버전에 따라 달라질 수 있다.

코드와 이용 조건

코드 원문은 사용자가 제공한 자료이며 저작자 표기를 유지했다. 설명을 추가해도 원본의 사용 조건은 바뀌지 않는다. 공개 배포와 판매에 필요한 권리는 별도로 확인한다. 본문은 이용 조건을 설명하며 법적 허가를 대신하지 않는다.

전집 길잡이 1

1권 | 처음 배우는 나비 로봇

01 전체 제작 지도 / 02 기계·전기·소프트웨어의 역할 / 03 단위와 기초 계산 / 04 준비물과 사양 기록

PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

2권 | CAD에서 Blender 입체 부품까지

05 원본 보관과 도면 조사 / 06 Blender 가져오기와 크기 맞추기 / 07 열린 윤곽·겹친 점 복구 / 08 구멍을 보존하고 면 만들기 / 09 두께·노멀·출력 메시 검사 / 10 낱개 STL과 전체 STL

PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

전집 길잡이 2

3권 | 출력과 날개 기구 조립

11 슬라이싱과 소형 부품 출력 / 12 끼워맞춤 시험편과 치수 기록 / 13 혼·링크·날개 조립 / 14 날개막·질량·무게중심

PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

4권 | 전원·무선·배선

15 제공된 배선도 읽기 / 16 전원 분배와 보드 차이 / 17 RC·PPM과 채널 설정 / 18 LED·신호 레벨·배선 점검

PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

전집 길잡이 3

5권 | Arduino 코드 전체 해설

19 설치·업로드와 최소 시험 / 20 변수·함수·조건문의 기초 / 21 입력 필터와 채널 지도 / 22 사인파와 실제 속도 / 23 활공·퍼칭·상태 유지 / 24 좌우 혼합·끝값 제한 / 25 신호 상실과 재시작 / 26 코드에 추가할 기능과 주석 방식

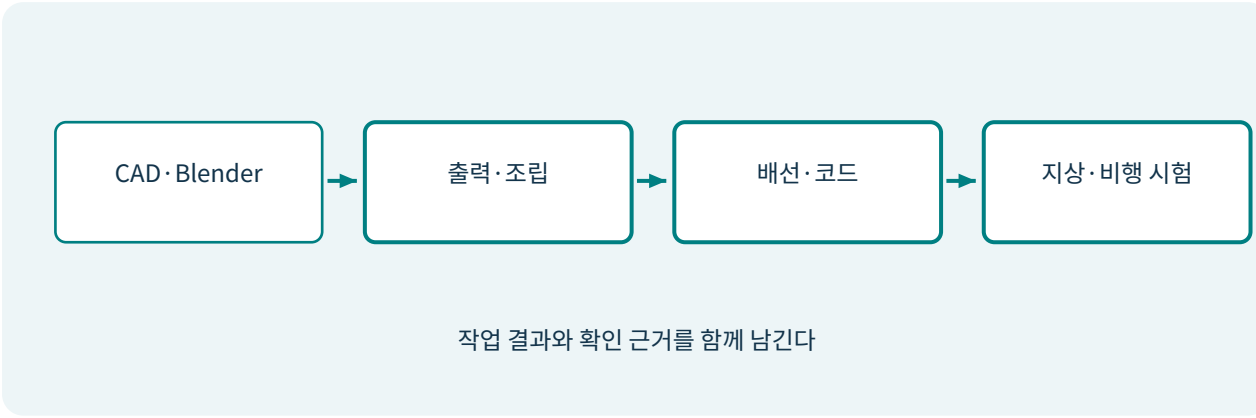
PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

6권 | 지상 검증·비행 시험·문제 해결

27 지상 단계별 시험 / 28 비행을 이해하는 최소 원리 / 29 첫 비행 준비와 기록 / 30 증상별 문제 해결 / 31 정비·개정·공유

PDF 책갈피에서 장 제목을 선택하면 해당 장으로 이동할 수 있습니다. 각 장은 그림 → 원리 → 작업 → 확인 순서로 읽습니다.

01 전체 제작 지도



이 책의 목표

CAD의 평면 윤곽을 출력 가능한 입체 부품으로 바꾸고, 디지털 서보 두 개와 PPM 수신기, Pro Micro 계열 보드로 날갯짓을 만드는 과정을 연결한다. 전자와 모델링을 처음 배우는 독자가 작업 순서와 확인 이유를 함께 이해하도록 구성했다. 지상에서 날개가 움직이는 것과 자유비행에 성공하는 것은 별도의 검증 단계다.

확인된 자료와 확인하지 못한 자료

사용자가 제시한 S-DiY 사인파 코드 전체와 wirring.jpg 배선 이미지를 분석 대상으로 삼았다. 파일 관리자 화면에는 Untitled3.dxf와 복구결과 폴더가 보이지만 DXF의 실제 내용은 제공되지 않았다. 그러므로 부품 개수, 구멍 지름, 날개 치수, 질량, 링크 길이, 실제 복구 성공 여부는 이 책에서 확정하지 않는다.

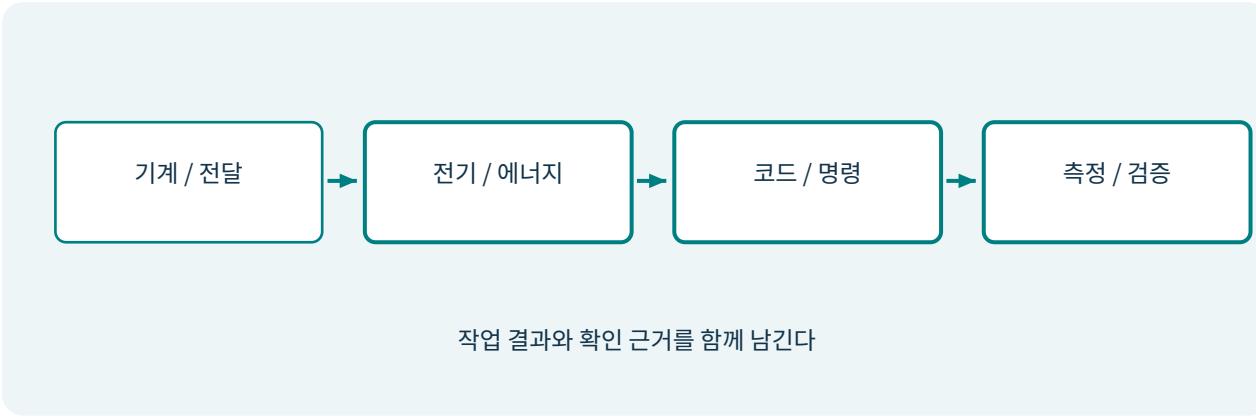
작업 순서

원본 보관 → CAD 단위와 윤곽 조사 → Blender 가져오기 → 크기 검증 → 구멍과 면 복구 → 두께 부여 → STL 저장 → 슬라이싱 → 출력 검사 → 서보 중립 맞춤 → 기구 조립 → 배선 → 입력 채널 확인 → 코드 시험 → 지상 시험 → 비행 조건 검토 순서로 진행한다.

사용 방법

각 장의 그림을 먼저 보고 설명과 절차를 읽는다. 장 끝의 확인 과제를 통과한 뒤 다음 단계로 간다. 수치가 예시라고 표시되면 실제 설계값으로 받아들이지 않는다. 원본 코드의 값과 교육용으로 낮춘 시험값을 기록에서 분리한다.

02 기계·전기·소프트웨어의 역할



기계가 하는 일

몸체는 모터와 전자부품을 지지하고, 혼은 서보축의 회전을 전달하며, 링크는 그 운동을 날개 뿌리로 전달한다. 날개막과 뼈대는 공기와 상호작용한다. 같은 서보 각도라도 혼 반지름과 링크 위치가 달라지면 날개 운동이 달라진다.

전기가 하는 일

배터리는 에너지원이며 레귤레이터나 BEC는 필요한 전압을 만든다. 모터 전원선은 구동 전류를, 신호선은 위치 명령을 운반한다. GND는 신호의 기준점이다. 신호가 정상이어도 모터 전압이 내려가면 떨림과 보드 재시작이 생길 수 있다.

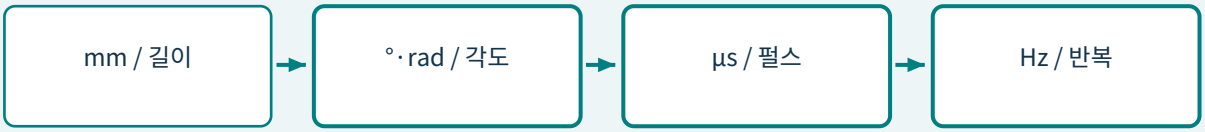
프로그램이 하는 일

무선 채널을 읽고, 모드를 선택하고, 반복 운동을 계산한 뒤 각 서보의 목표 위치를 출력한다. 프로그램이 명령한 위치에 실제 서보가 도착했는지는 이 배선으로 측정하지 않는다. Servo.read()도 실제 센서값이 아니라 최근 명령값을 반환한다. [S2]

확인 과제

종이에 신호 흐름과 전원 흐름을 다른 색으로 그린다. 서보가 안 움직일 때 기계 걸림, 전압, 신호를 각각 어떻게 확인할지 적는다. 원인을 한 번에 하나씩 분리하는 습관이 제작 시간을 줄인다.

03 단위와 기초 계산



작업 결과와 확인 근거를 함께 남긴다

길이·각도·시간

1 cm는 10 mm, 1 m는 1000 mm다. 각도 180°는 π rad이며 사인 함수는 라디안을 사용한다. 1 ms는 1000 µs다. RC 채널의 1500이라는 숫자는 보통 시간 정보를 표현하며 1500°라는 뜻이 아니다.

왕복 운동의 말

진폭 A는 중심에서 한쪽 끝까지의 크기다. 중심 90°, 진폭 65°라면 순수 명령 범위는 25°부터 155°이며 양 끝 차이는 130°다. 이 값은 서보 명령 좌표다. 실제 날개 각도는 링크 기하와 조립 방향으로 결정된다.

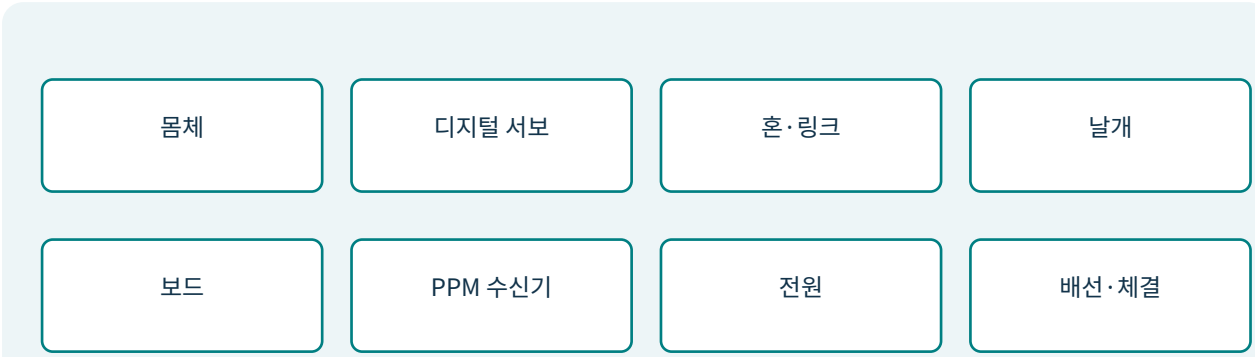
주파수와 주기

초당 반복 횟수를 f Hz, 한 번 반복하는 시간을 T s라 하면 T=1/f다. 2 Hz는 한 주기가 0.5초다. 프로그램 계산 횟수, 서보 명령 갱신 횟수, 실제 날개 왕복 횟수는 서로 다를 수 있다.

예제와 연습

도면의 40 mm 구간이 Blender에서 0.04 m라면 길이가 일치한다. 40 m라면 1000배 차이다. 1.5 mm 두께에 0.2 mm 레이어 높이를 쓰면 정확히 정수 층이 되지 않으므로 슬라이서가 실제 상하면을 어떻게 구성하는지 확인한다. 두께 1.5 mm는 대화에서 사용한 작업 예시이며 원본 설계의 승인 치수가 아니다.

04 준비물과 사양 기록



기계 부품

몸체와 브래킷 출력물, 위치제어 디지털 서보 두 개, 해당 서보에 맞는 혼과 나사, 축·부싱·링크, 날개 뼈대와 막, 배터리 고정부가 필요하다. 나사 규격과 수량은 실제 도면 및 서보 규격을 조사해 채운다. 연속회전 서보는 목표 각도를 지정하는 이 코드의 대체품이 아니다.

전자 부품

PPM 출력 가능한 수신기와 대응 조종기, 실제 사양이 확인된 ATmega32U4 보드, 전압과 전류가 맞는 전원 구성, 전원 차단 스위치, 커넥터와 전선이 필요하다. D6의 외부 LED에는 전류 제한 저항을 사용한다. 큰 조명 부하는 GPIO에서 직접 구동하지 않고 드라이버를 따로 설계한다.

공구와 소프트웨어

캘리퍼스, 저울, 멀티미터, 납땜 공구, 작은 드라이버, USB 데이터 케이블을 준비한다. CAD 프로그램, Blender, 슬라이서, Arduino IDE와 호환 보드 패키지 및 라이브러리를 사용한다. 소프트웨어 버전과 설정을 작업일지에 기록한다.

구매 전 확인표

서보: 전압 / 질량 / 속도 / 토크 / 입력 펄스 / 허용 갱신율. 보드: MCU / 5V 또는 3.3V / 클록 / RAW 회로. 수신기: PPM 채널 수 / 출력 전압 / 신호 상실 동작. 배터리: 셀 수 / 최대 충전전압 / 용량 / 방전 허용전류. 현재 자료만으로 특정 모델을 구매 확정하지 않는다.

05 원본 보관과 도면 조사



폴더 구성

원본은 00_original, 도면 정리는 01_cad, Blender는 02_blender, STL은 03_stl, 슬라이서는 04_slice, 조립 사진은 05_build, 코드는 06_firmware, 시험 기록은 07_test처럼 구분한다. 이는 권장 관리 예시다. 복구 파일은 원본과 다른 이름으로 보관한다.

CAD에서 조사

DXF를 CAD에서 열어 모델 공간과 레이아웃을 구분한다. 외곽선, 안쪽 구멍, 치수선, 문자, 중심선, 블록을 레이어별로 조사한다. 치수 보조선을 실제 절단 윤곽으로 착각하지 않도록 출력 대상 레이어를 따로 복사한다.

실측 기준 만들기

도면의 알려진 두 점 사이 거리를 측정한다. 화면을 크게 확대했다고 실제 길이가 커지는 것은 아니다. 단위 정보가 불확실하면 기준 길이와 원본 작성자 정보를 우선한다. 알려진 치수가 없으면 임의로 전체를 축소해 실제 크기라고 확정하지 않는다.

저장과 완료 조건

겹친 선, 아주 짧은 조각, 열린 끝점을 확인하고 복사본에서 정리한다. 블록을 분해할 때는 형상이 변하지 않았는지 비교한다. 내보내기 DXF 버전은 가져오기 도구와 맞춘다. 완료 조건은 기준 길이, 대상 레이어, 원본 보존 위치를 설명할 수 있는 것이다.

06 Blender 가져오기와 크기 맞추기



표시 단위와 실제 길이를 각각 검증

가져오기 경로

Blender 4.0 계열에서는 설치 구성에 따라 AutoCAD DXF 가져오기 애드온을 활성화해야 한다. Edit > Preferences > Add-ons에서 DXF를 검색하고 제공되는 항목을 확인한다. File > Import에서 DXF가 없으면 버전과 설치된 도구를 확인한다. 새 버전의 확장 설치 화면은 다를 수 있다.

장면과 단위

Scene Properties의 Units에서 Metric을 선택하고 표시 단위를 정한다. Unit Scale과 표시 단위, 물체의 실제 크기는 구분한다. N 패널 Item의 Dimensions로 기준 길이를 확인한다. 도면 기준 40 mm가 40 m로 들어오면 해당 경우에 한해 배율 0.001로 보정한다.

조작 순서

오브젝트 선택 → N 패널 치수 확인 → 필요한 배율 S 입력 → 기준 길이 재확인 → Object Mode에서 Ctrl+A > Scale. G X 10mm처럼 단위를 붙여 이동하면 의도를 명확히 할 수 있다. Edit Mode에서 모든 점을 이동하면 오브젝트 원점은 따라 움직이지 않을 수 있다.

불러온 데이터 종류

Curve, Mesh, Empty, Text 중 어떤 형식인지 Outliner와 속성으로 확인한다. 곡선은 필요할 때 Object > Convert > Mesh로 복사본을 변환한다. 선이 보인다는 이유만으로 면이 만들어진 것은 아니다. 카메라·조명·문자 같은 보조 물체를 출력 컬렉션에서 제외한다.

07 열린 윤곽·겹친 점 복구



열린 끝점과 안쪽 구멍

끝점 복구 / 구멍 보존

문제를 먼저 분류

면이 안 생기면 열린 끝점, 중복 선, 서로 교차하는 윤곽, 다른 높이에 놓인 점, 지나치게 작은 틈을 차례로 조사한다. 한 번에 모든 점을 크게 병합하면 구멍과 얇은 부분을 잃을 수 있다. 항상 복사본에서 작은 허용거리로 시작한다.

끝점을 잇는 방법

Mesh Edit Mode에서 끝점 두 개를 선택하고 F로 연결선을 만든다. Merge by Distance는 같은 위치에 가까이 겹친 점을 합칠 때 쓴다. 단순히 멀리 떨어진 끝점을 큰 병합 거리로 끌어당기는 복구는 원래 윤곽을 왜곡할 수 있다.

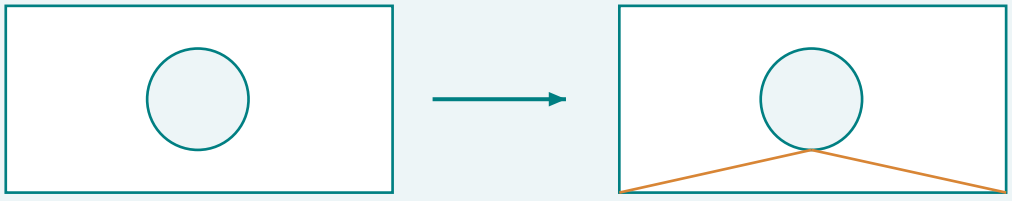
중간 선의 의미

보이는 선은 정상적인 삼각분할 경계일 수도 있고, 중복 면·열린 틈·내부 면일 수도 있다. 선이 있다는 사실만으로 불량이라고 판단하지 않는다. Wireframe, Face Orientation, 면 선택과 단면, 슬라이서 미리보기로 실제 면 연결을 조사한다. 필요 없는 내부 경계만 Dissolve Edges로 정리한다.

완료 조건

외곽과 구멍 윤곽이 끊기지 않고 자기 교차하지 않아야 한다. 평면 부품이면 같은 평면인지 확인한다. 삭제 전에 면이 사라질지 미리 판단한다. Delete Edges와 Dissolve Edges는 결과가 다르므로 실수하면 즉시 Undo하고 구조를 다시 본다.

08 구멍을 보존하고 면 만들기



외곽 + 내부 구멍　　구멍 둘레를 유지한 면

구멍은 빈 공간이다

외곽 테두리와 안쪽 원을 한 번에 F로 채우면 기대한 관통 구멍이 보존되지 않을 수 있다. 일반적인 한 장의 n-gon은 독립된 내부 구멍 고리를 그대로 표현하는 방식이 아니다. 자동 채움 결과는 반드시 눈으로 확인한다.

방법 A: 평면 곡선

복사본의 Curve에서 2D와 Fill 설정을 활용한다. 안쪽 고리 방향과 닫힘 상태에 따라 채움이 달라질 수 있으므로 변환 전후를 비교한다. 면이 올바르면 Mesh로 변환하여 관통 구멍과 면 방향을 조사한다. 가져온 데이터에 따라 이 방법이 적합하지 않을 수 있다.

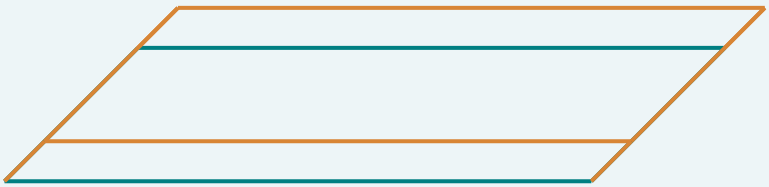
방법 B: 면과 절삭체

먼저 외곽을 면과 두께가 있는 몸체로 만든다. 구멍 위치에 원기둥 절삭체를 두고 Boolean Difference로 뺀다. 절삭체가 두께를 충분히 관통하게 하고 반지름·좌표를 입력한다. 결과 확인 후 Modifier를 적용하고 절삭체는 내보내기에서 제외한다.

브리지 사용의 조건

Bridge Edge Loops는 대응 가능한 경계 사이를 잇는 도구다. 구멍마다 무조건 실행하는 복구 명령이 아니다. 정점 수와 선택 범위를 확인한다. 면 생성 뒤 노멀과 겹침을 검사하고, 원본 기준 구멍 중심 및 지름이 유지됐는지 측정한다.

09 두께·노멀·출력 메시 검사



평면 → 두께 있는 부피 / 1.5 mm는 작업 예시

두께를 주는 두 가지 방법

평면 Mesh의 면을 선택해 E Z 1.5mm로 축 방향 돌출하거나 Solidify Modifier로 두께를 줄 수 있다. 두 방법을 동시에 중복 적용하지 않는다. 1.5 mm는 예시다. 부품의 하중, 재료, 출력 방향, 원본 치수에 따라 실제 두께를 정한다.

기준면과 Offset

Solidify의 Offset은 원래 면을 기준으로 어느 쪽에 두께를 배분할지 정한다. 두께가 늘어나면서 결합 기준면이나 서보 간격이 바뀌는지 조사한다. 기울어진 면이나 변형된 스케일은 예상과 다른 두께를 만들 수 있어 Scale 적용과 실제 측정을 함께 한다.

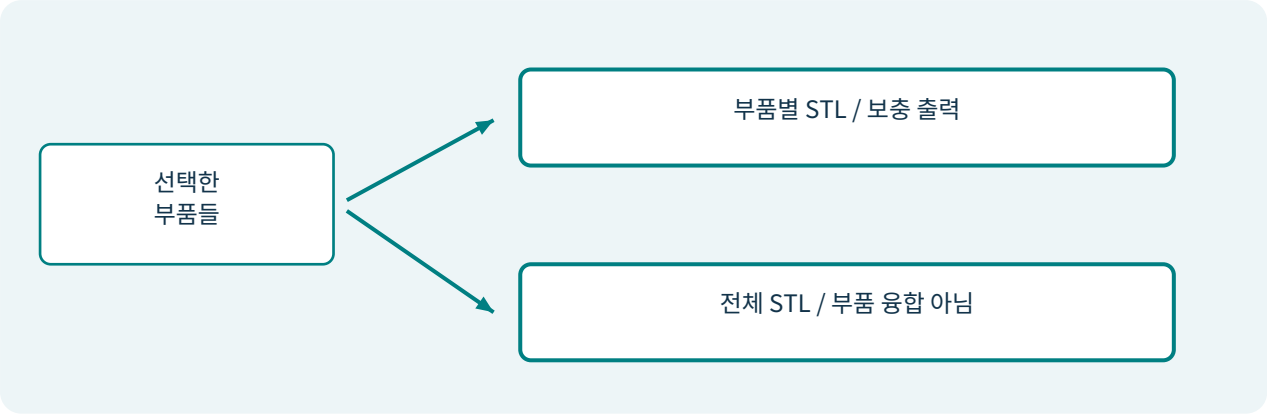
면 방향과 닫힘

닫힌 부피 Mesh에서는 Edit Mode A 후 Shift+N으로 바깥 방향 재계산을 시도하고 Face Orientation으로 검사한다. Non-manifold 선택으로 열린 경계를 찾되 결과의 원인을 직접 확인한다. 평면 모델 단계의 열린 경계는 정상일 수 있지만 최종 출력 부피의 의도하지 않은 틈은 수정 대상이다.

완료 조건

전체 두께와 구멍 지름을 재측정하고 내부에 겹친 면이 없는지 확인한다. 슬라이서에서 층별로 관통 구멍이 이어지는지, 얇은 벽이 사라지지 않는지 확인한다. STL을 다시 가져와 검사하는 왕복 검증으로 내보내기 실수를 잡는다.

10 낱개 STL과 전체 STL



낱개 저장

Object Mode에서 출력할 부품 하나만 선택한다. File > Export > STL에서 Selection Only를 활성화한다. 부품 이름과 개정 번호를 파일명에 넣고 저장한다. 다음 부품도 반복한다. Batch 옵션이 있는 버전은 각 오브젝트를 따로 저장할 수 있지만 결과 파일 수와 내용을 확인한다. [S5]

전체 저장

출력할 부품들만 선택하고 Selection Only를 켠 상태에서 한 STL로 내보낸다. 이때 여러 부품이 한 파일에 담길 뿐 실제로 융합되지는 않는다. Ctrl+J도 오브젝트 결합이며 겹친 부피를 하나의 솔리드로 만드는 Boolean Union과 다르다.

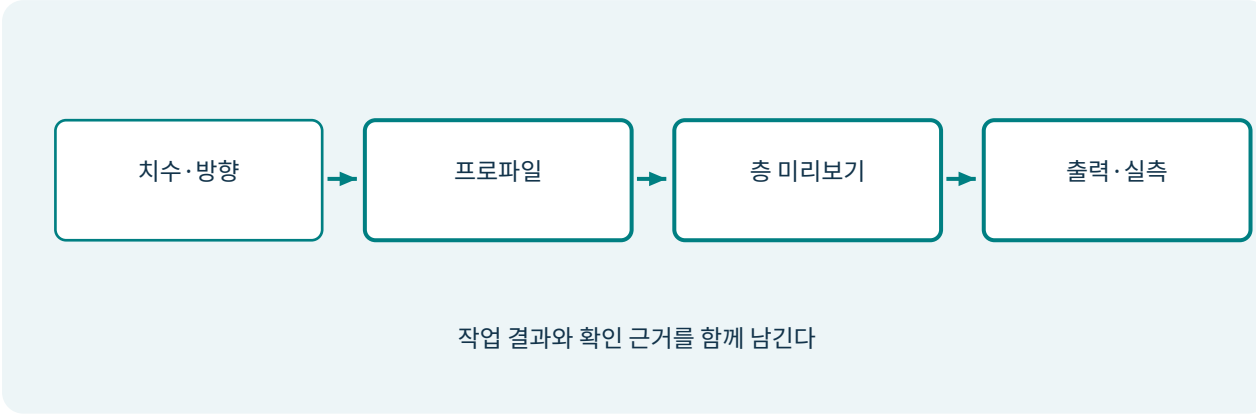
단위 검증

STL 자체에는 명확한 물리 단위 정보가 없다. exporter의 Scale과 Scene Unit 설정을 기록하고, 슬라이서에 가져온 후 기준 길이가 같은 mm인지 확인한다. 표시 단위만 보고 크기가 맞다고 단정하지 않는다. 버전별 옵션 차이를 피하려면 10 mm 기준 큐브를 함께 시험한다.

선택 기준

부품별 방향·재료·보충 출력이 필요하면 낱개 STL이 편하다. 같은 배치를 전달하려면 전체 STL을 쓸 수 있다. 조립체 3MF 또는 슬라이서 프로젝트는 배치와 출력 설정 보존에 유리하지만 다른 프로그램에서의 호환성은 확인한다. 원본 .blend도 별도로 남긴다.

11 슬라이싱과 소형 부품 출력



출력 전에

슬라이서에서 프린터·노즐·재료 프로파일을 실제 장비와 맞춘다. STL 치수, 바닥 접촉, 최소 벽, 구멍과 움직이는 틈을 확인한다. 모델을 연 상태와 G-code를 내보낸 상태는 다르다. 최종 G-code 미리보기를 확인한다.

방향과 적층

나사 주변이나 링크 뿌리가 층 사이로 벌어지는 방향인지 검토한다. 평평하게 눕히면 서포트가 줄지만 모든 부품의 최적 강도를 보장하지 않는다. 주요 인장·굽힘 방향과 적층 경계를 함께 판단한다. 얇은 부품은 첫 층 과압착으로 구멍과 외곽이 퍼질 수 있다.

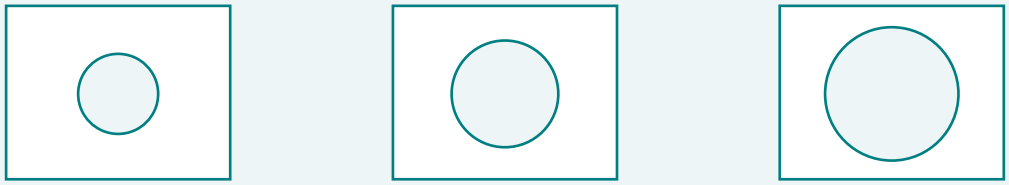
예시 설정

0.4 mm 노즐과 0.2 mm 층높이는 시험 시작 예시이며 정답이 아니다. 벽 수, 상하면 층, 인필은 실제 형상에 맞춘다. 1.5 mm 같은 얇은 부품에서는 인필보다 외벽과 상하면이 대부분을 차지할 수 있다. 유기형 서포트는 필요한 돌출부에만 사용하고 축구멍 표면 손상을 확인한다.

출력 후 검사

충분히 식힌 뒤 분리하고 버·실·서포트를 제거한다. 구멍 가공은 기준 치수를 확인하며 조금씩 한다. 층분리와 균열이 있으면 조립으로 감추지 않는다. 출력물 질량을 재고 좌우 짝의 차이를 기록한다. 결합이 너무 빡빡하면 설계 여유와 프린터 보정 문제를 구분한다.

12 끼워맞춤 시험편과 치수 기록



여유 A　　여유 B　　여유 C

시험편을 먼저 만드는 이유

실제 프린터의 구멍과 축 오차는 재료 수축, 압출, 방향, 냉각에 따라 다르다. 전체 몸체를 반복 출력하기 전에 축·구멍·서보 고정부 일부를 시험편으로 만들어 재료와 시간을 줄인다.

허용차 표 만들기

실제 축을 측정해 기준으로 삼는다. 예를 들어 측정 축 지름에 대해 구멍을 +0.1, +0.2, +0.3 mm씩 바꾼 시험편을 만들 수 있다. 이 증분은 교육 예시다. 손으로 잘 도는 유격과 구조적으로 필요한 고정 정도를 확인한 뒤 실제 값을 선택한다.

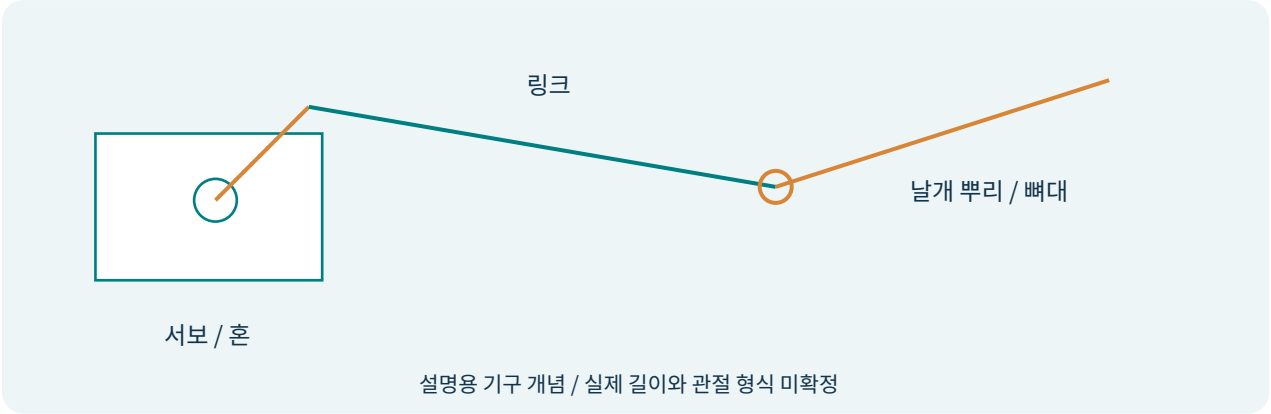
서보 고정

서보 본체 치수뿐 아니라 귀의 두께, 나사 위치, 케이블 출구, 혼 회전 공간을 측정한다. 나사를 과하게 조여 플라스틱을 압궤하지 않는다. 진동으로 빠지지 않는 고정과 정비를 위한 분해 가능성을 함께 고려한다.

완료 조건

캘리퍼스 실측값, CAD 목표값, 출력 결과값, 선택한 여유값을 한 줄에 기록한다. 한 번 맞았던 치수라도 재료와 노즐 또는 출력 방향이 바뀌면 시험편을 다시 확인한다.

13 혼·링크·날개 조립



설명용 기구 개념 / 실제 길이와 관절 형식 미확정

무전원 기구 점검

전자부품을 넣기 전에 축과 링크가 부드럽게 움직이는지 확인한다. 서보축을 억지로 돌려 검사하지 말고 링크를 분리해 각각의 마찰을 조사한다. 나사를 조이면 축이 잠기는 구조인지도 확인한다.

중립에서 조립

날개 링크를 떼고 서보 두 개에 중립 명령을 준다. 전원을 차단한 뒤 기준 자세에 맞춰 혼을 장착한다. 혼 이빨 간격 때문에 정확히 맞지 않으면 작은 전자 트림으로 보정하되 큰 기계 오차를 코드로 감추지 않는다.

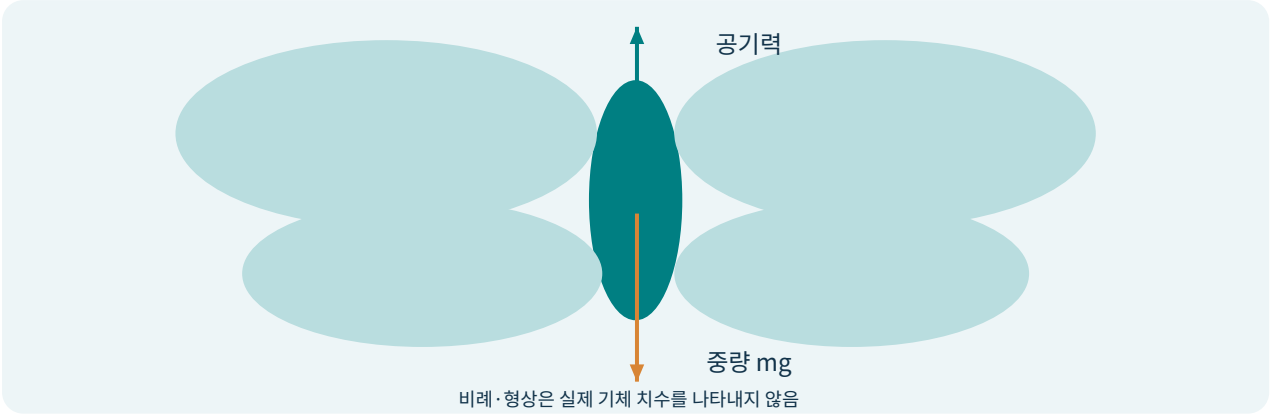
운동 전달의 원리

작은 각도에서 혼 끝 변위는 대략 r×θ이다. 여기서 r은 혼 반지름, θ는 라디안이다. r=10 mm, θ=10°이면 약 1.75 mm라는 근사 예시가 된다. 큰 각도에서는 링크의 비선형성과 간섭을 직접 계산하거나 CAD로 확인해야 한다.

사점과 간섭

링크가 일직선에 가까워져 힘 전달이 나빠지는 자세, 혼이 몸체를 치는 자세, 전선이 끼는 자세를 찾는다. 작은 진폭부터 늘리면서 양 끝에서 여유를 확인한다. 전원 차단 후에도 기계적으로 걸리지 않아야 다음 단계로 넘어간다.

14 날개막·질량·무게중심



날개는 출력물 전체가 아니다

몸체를 3D 프린터로 만들 수 있어도 날개 전체를 무겁게 출력하면 비행 조건이 달라진다. 원본 설계가 사용하는 뼈대와 막 재료를 먼저 확인한다. 막 장력과 휨은 공기력과 서보 부하를 함께 바꾼다.

질량 예산

몸체, 좌우 서보, 날개, 배터리, 수신기, 보드, 배선, 체결물의 질량을 각각 재고 합한다. m_total=각 부품 질량의 합이다. 여유 없이 부품을 추가하면 날갯짓은 유지되어도 비행 능력이 떨어질 수 있다.

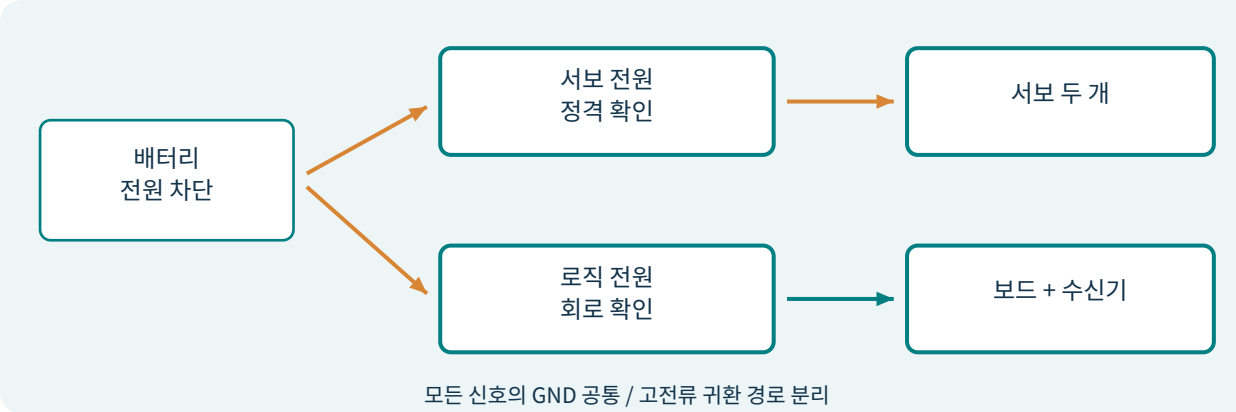
무게중심

한 축의 무게중심은 x_cg=Σ(m_i x_i)/Σm_i로 구한다. 같은 기준점에서 위치를 재야 한다. 배터리 이동은 큰 영향을 줄 수 있다. 목표 무게중심은 날개 형상과 기체의 안정 특성에 따라 달라지므로 임의의 몇 %를 이 기체의 정답으로 쓰지 않는다.

완료 조건

좌우 날개 형상과 질량 차이, 비틀림, 축 유격, 배터리 고정 상태를 기록한다. 날갯짓 영상으로 실제 각도와 흔들림을 비교한다. 지상에서 대칭으로 움직이는 것만으로 공중 조향 방향을 확정하지 않는다.

15 제공된 배선도 읽기



모든 신호의 GND 공통 / 고전류 귀환 경로 분리

원본 그림의 성격

다음 페이지의 사진은 사용자가 제공한 참고 배선도다. 선 색은 그림의 범례이며 실제 제품의 배선 색을 보증하지 않는다. 위쪽의 6V·8.4V 문구는 해당 제작 구성을 위한 표기다. 모든 보드·서보·수신기에 적용 가능한 전원 규격이 아니다.

신호 핀

제공 코드에서 PPM 입력은 D3, servoLeft는 D9, servoRight는 D10이다. D6는 추가 LED 출력이다. 원본 그림의 좌우 명칭은 코드와 일치하지 않고 오른쪽 그림의 베트남어·영어 표기도 서로 맞지 않는다. 케이블에 D9·D10 표찰을 붙여 혼동을 줄인다.

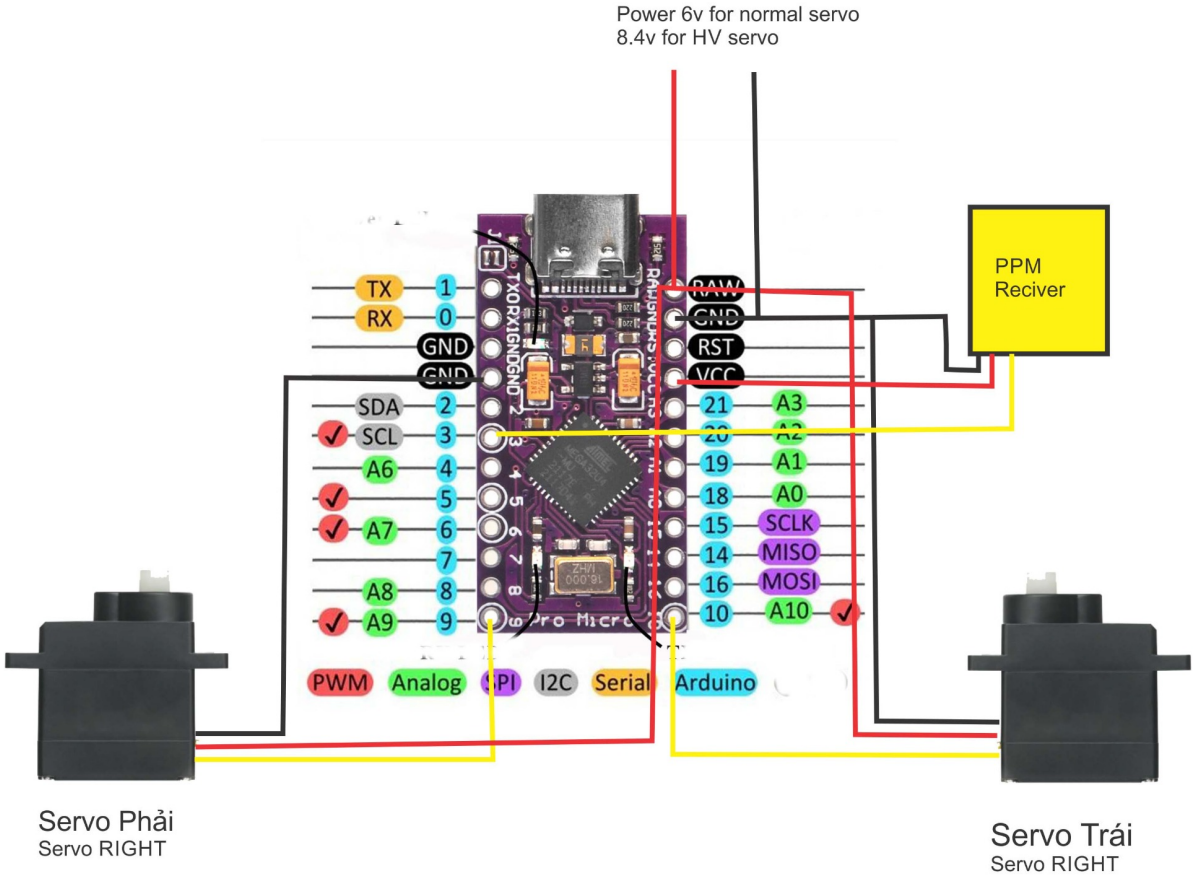
전원 경로

그림에서 외부 전원은 서보 양극과 보드 RAW로 갈라지고 GND를 공통으로 잇는다. 수신기 양극은 보드 VCC에 연결되어 있다. 실제 보드와 수신기의 전압 및 전류 허용치가 이 연결을 지원할 때만 성립한다. 단지 그림과 닮은 USB-C 보드라고 같은 회로로 가정하지 않는다.

접속표 작성

연결하기 전에 출발 단자 / 도착 단자 / 기능 / 예상 전압 / 확인 결과를 표로 적는다. PPM 신호가 수신기 몇 번 포트에서 나오는지도 제품 매뉴얼에서 확인한다. PWM 채널 포트 한 개를 PPM 포트라고 생각해 연결하지 않는다.

참고 그림 | 사용자 제공 배선도



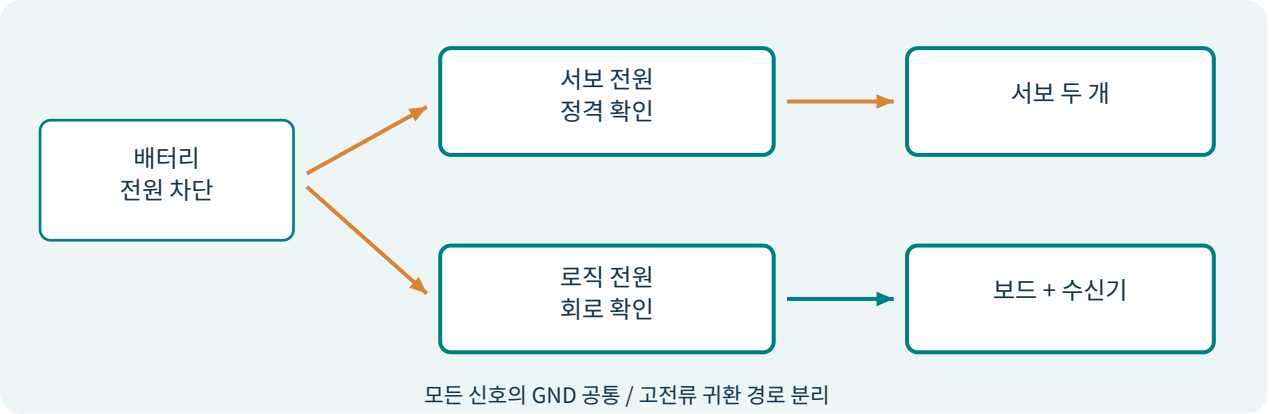
사진의 연결 관계

수신기 신호 → D3, 코드의 왼쪽 서보 → D9, 오른쪽 서보 → D10. 전원은 보드와 서보로 나뉘며 수신기는 보드 VCC와 연결된 것으로 읽힌다. 원본 그림의 좌우 표기 오류와 보드 전원 회로의 불확실성을 15~16장에서 설명했다.

이 그림을 사용할 때

전원 정격 확인 전에는 사진과 똑같이 결선해 전원을 넣지 않는다. 6V와 8.4V는 모든 장치에 대한 공통 허용전압이 아니다. USB-C 보드의 실제 모델명과 회로도를 먼저 확인한다. 이 사진에는 D6 LED 회로가 나타나 있지 않다.

16 전원 분배와 보드 차이



RAW와 VCC

기존 SparkFun Pro Micro의 RAW는 레귤레이터 경로이고 VCC는 보드 동작 전압 레일이다. [S1] 그러나 USB-C 계열에는 다른 전원 구성이 존재한다. 공식 Qwiic Pro Micro 자료도 점퍼 위치에 따라 전원 경로가 달라짐을 설명한다. [S6] 실제 제품 회로도 없이는 8.4V RAW 연결을 승인할 수 없다.

권장 검토 구조

배터리 → 스위치 → 모터용 전압 조정부 → 서보 두 개, 그리고 별도 로직 전원 → 보드와 수신기의 구조를 먼저 검토한다. 같은 레귤레이터를 공유해도 전압·순간전류·노이즈 조건을 충족해야 한다. 각 GND를 기준점에서 연결하고 서보 전류가 보드의 얇은 배선을 통과하지 않게 한다.

전류 계산 예시

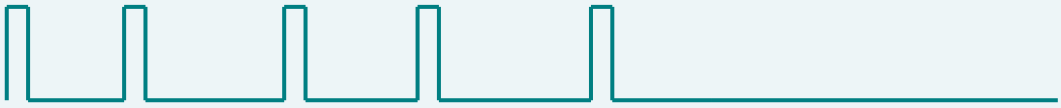
서보 정지전류가 모델 자료에서 각각 1 A라면 두 개 합계는 2 A다. 여기에 수신기와 보드 전류 및 순간 변화 여유를 검토한다. 이 숫자는 실제 서보의 값이 아니다. 배터리 용량만 크다고 전압 강하와 전선 과열 문제가 해결되는 것은 아니다.

전원 투입 절차

배터리를 떼고 극성과 단락을 확인한다. 가능하면 전류 제한 가능한 시험 전원으로 로직부터 점검한다. 부하 상태 전압을 측정하며 서보 한 개, 두 개, 링크 연결 순으로 확장한다. USB와 외부 전원을 동시에 연결하는 조건은 보드의 전원 회로를 확인한 후 사용한다.

17 RC·PPM과 채널 설정

PPM: 여러 채널의 시간 간격을 한 신호선에 표현



채널 간격과 프레임 구분 간격 / 파형은 설명용

무선과 유선 신호의 구분

조종기와 수신기는 무선으로 통신한다. 수신기와 보드는 PPM이라는 유선 신호로 연결된다. Arduino가 조종기의 전파를 직접 수신하는 구조가 아니다. PPM은 채널 정보를 연속된 시간 간격으로 표현한다.

라이브러리 조건

PPMReader는 같은 이름의 구현과 포크가 있을 수 있다. 원본 작성자의 저장소와 버전을 기록한다. 조사한 Nikkilae 구현은 인터럽트 지원 핀과 1부터 시작하는 채널 번호를 사용한다. [S3] 실제 사용 라이브러리의 기본 타이밍·입력 극성·지원 보드를 확인한다.

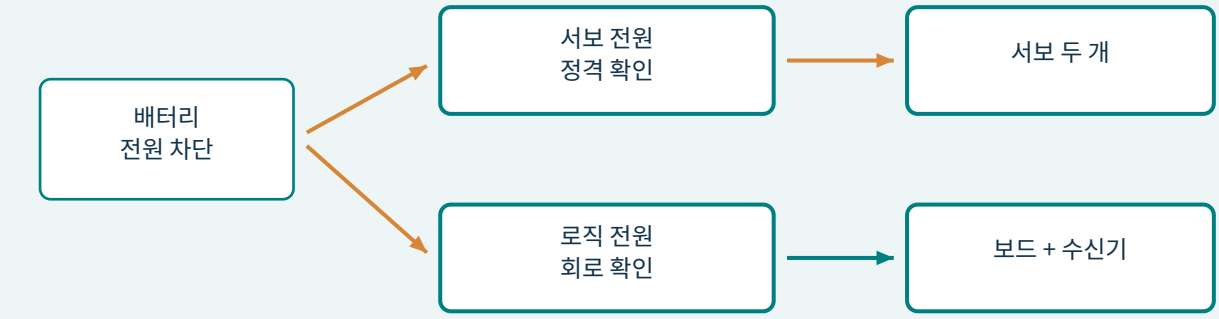
채널 관찰

서보를 분리한 상태에서 채널 출력 예제로 조종기 스틱과 스위치를 하나씩 움직인다. CH1~CH8의 최소·중앙·최대, 반전, 스위치 단계, 조종기 전원 OFF 시 출력을 적는다. 1000/1500/2000은 흔한 예시이며 실제 endpoint를 측정한다.

시작 상태

스로틀을 낮춘 상태로 켜고 수신기의 연결 표시를 확인한다. 수신기가 연결 안 됐을 때 1500을 계속 내보내면 원본 코드는 날갯짓으로 해석할 수 있다. 수신기 페일세이프 채널값과 프로그램의 시작 허가 조건을 별도로 검토한다.

18 LED·신호 레벨·배선 점검



모든 신호의 GND 공통 / 고전류 귀환 경로 분리

LED 추가

D6 → 전류 제한 저항 → LED → GND 순으로 연결할 수 있다. 극성을 확인한다. R=(V_pin−V_f)/I로 계산한다. 5V 출력, 순방향 2V, 목표 5mA라면 600Ω이라는 계산 예시이며 가까운 표준값 680Ω을 검토할 수 있다. 실제 핀 전류와 LED 정격을 확인한다.

신호 레벨

3.3V 보드에 5V PPM 신호를 직접 넣어도 된다고 가정하지 않는다. MCU 핀 정격과 수신기 출력 형식을 확인하고 필요하면 레벨 변환을 사용한다. 서보 역시 3.3V 신호를 인식하는지 사양에서 확인한다.

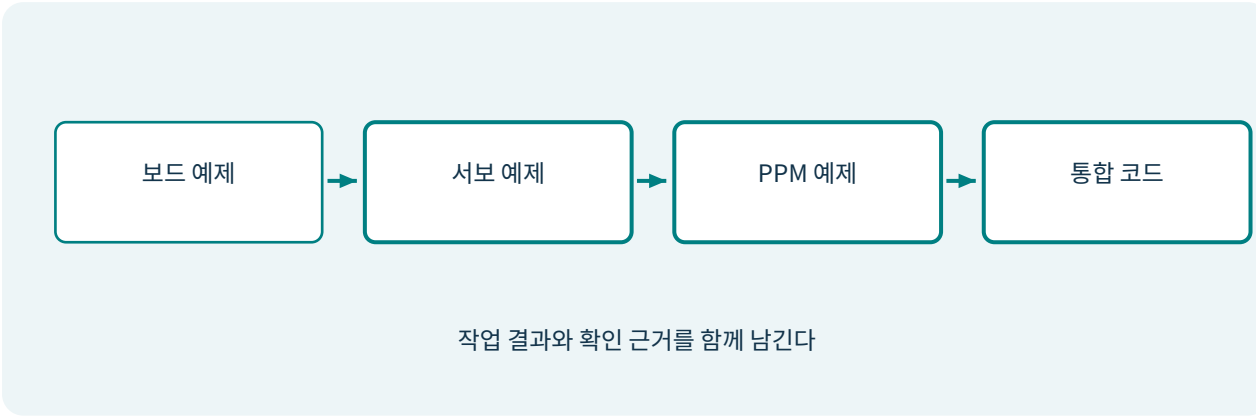
배선 마감

커넥터에 극성과 기능 표찰을 붙이고 납땜부를 절연한다. 반복 운동부 근처에 케이블 여유를 주되 링크에 걸리지 않게 고정한다. 얇은 신호선과 큰 모터 전류의 귀환 경로를 의도적으로 배치한다. 진동 시험 후 접촉 불량과 피복 쓸림을 다시 확인한다.

완료 조건

보드·수신기·서보 각 전원 핀의 실제 전압, 모터 동작 중 최저 전압, GND 연속성, 신호 핀 번호를 기록한다. D6의 LED가 켜진다고 PPM 수신이나 비행 준비가 자동으로 증명되지는 않는다.

19 설치·업로드와 최소 시험



환경 기록

Arduino IDE에서 실제 ATmega32U4 보드에 맞는 패키지·보드·프로세서·전압/클록·포트를 선택한다. USB-C 커넥터 모양만 보고 보드 종류를 선택하지 않는다. 데이터 전송 가능한 케이블을 사용한다. 업로드 오류 메시지를 원문 그대로 보관한다.

단계별 업로드

먼저 단순 LED 또는 시리얼 예제로 보드와 USB 통신을 확인한다. 다음은 서보 하나의 작은 범위 왕복, 다음은 PPM 채널 표시, 마지막은 통합 코드 순서다. 장치와 코드를 동시에 모두 바꾸면 고장 원인을 찾기 어렵다.

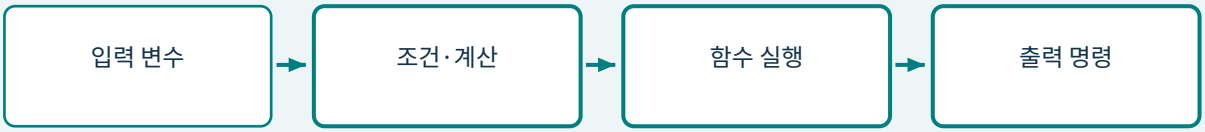
두 라이브러리

Servo.h는 서보 펄스를 만들고 PPMReader.h는 수신기 신호를 해석한다. include 파일을 찾을 수 없으면 라이브러리 설치 위치와 이름을 조사한다. 설치가 된 뒤에도 선택 보드와 컴파일이 호환되는지 확인한다. 사용한 라이브러리의 저장소·버전·라이선스를 기록한다.

시험 범위

이 책의 원본 스케치는 사용자가 제공한 코드를 그대로 수록한 학습 자료다. 이 작성 환경에는 Arduino CLI와 실제 보드가 없어 컴파일·업로드·동작 시험을 수행하지 못했다. 책 속 설명과 정적 검토는 실제 하드웨어 검증을 대체하지 않는다.

20 변수·함수·조건문의 기초



작업 결과와 확인 근거를 함께 남긴다

변수와 상수

const int centerAngle=90은 바뀌지 않는 정수 설정이다. float phase는 소수점 위상을 저장한다. bool isPerchingState는 참/거짓 상태를 저장한다. byte ch는 작은 정수이고 배열 lastChValues[8]은 여덟 칸이다. 배열 인덱스는 0부터 7까지다.

함수의 역할

setup()은 시작할 때 한 번 실행하고 loop()는 반복 실행한다. updateServos()처럼 이름 붙인 함수는 공통 작업을 묶는다. 함수 인자는 입력값이고 return은 결과를 돌려준다. void는 반환 결과가 없는 함수다.

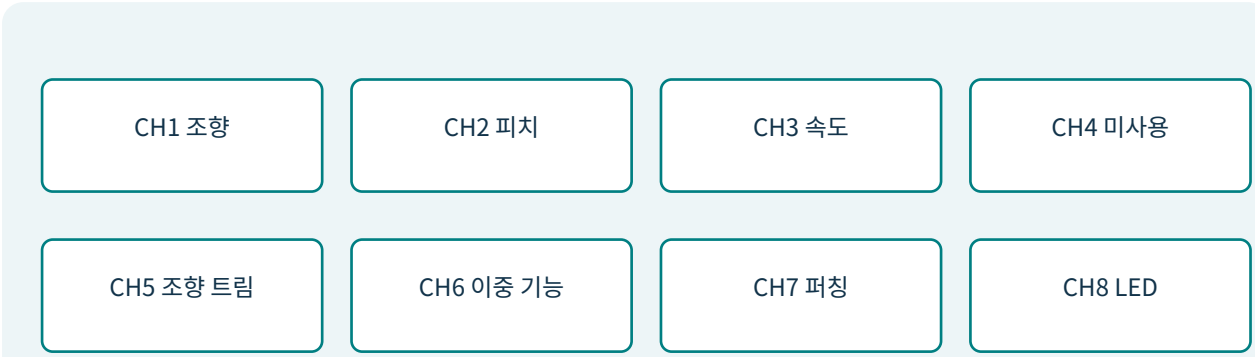
조건과 연산

if는 조건을 검사하고 else는 반대 경우를 처리한다. &&는 두 조건이 모두 참일 때, ||는 하나라도 참일 때를 뜻한다. =는 대입이고 ==는 같은지 비교다. (int)는 소수 부분을 버리는 정수 변환이며 반올림과 다르다.

map과 constrain

map은 한 숫자 범위를 다른 범위로 선형 변환하지만 입력을 자동 제한하지 않는다. Arduino의 정수 map은 소수점 결과를 보존하지 않는다. constrain은 최소·최대 사이로 제한한다. 범위 변환 전에 입력 유효성을 확인하는 이유가 여기에 있다.

21 입력 필터와 채널 지도



채널 배정

CH1 조향, CH2 엘리베이터, CH3 날갯짓 속도, CH4 미사용, CH5 조향 트림, CH6 피치 트림 및 퍼칭 범위, CH7 퍼칭 선택, CH8 LED다. 이 이름은 코드의 의도이고 실제 공중의 반응 방향은 기구와 공기력에 따라 시험해야 한다.

readPPMFiltered

readPPMFiltered(0)은 내부에서 ch+1을 사용해 실제 CH1을 읽는다. 이전값과 차이가 3보다 클 때만 배열 값을 갱신한다. 이동평균이 아니라 변화량 문턱 필터다. 예를 들어 이전 1500에 새 값 1502면 유지하고 1504면 갱신한다.

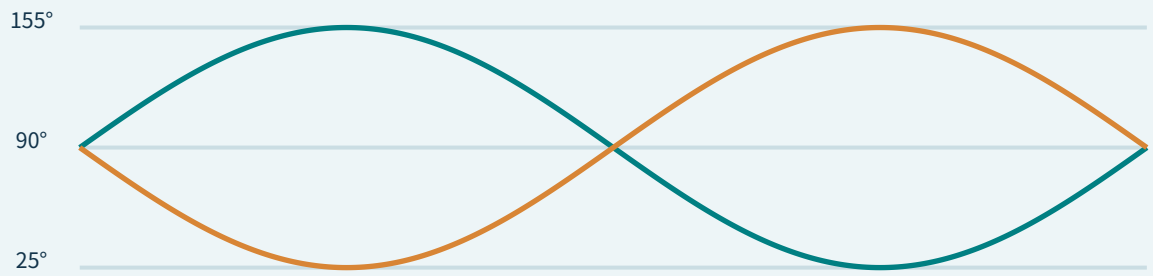
조향 계산

steering이 1500에서 10 넘게 벗어나면 900~2100을 −15~15로 map한다. 여기에 CH5의 −15~15 트림을 더한다. CH5가 1000~2000을 벗어나면 map 결과도 범위 밖으로 나갈 수 있다. 최종 서보 제한은 있으나 입력이 정상이라는 뜻은 아니다.

LED 필터

CH8>1550이면 카운터를 5까지 증가시키고 <1450이면 0까지 감소시킨다. 1450~1550에서는 유지한다. 출력은 카운터가 5일 때만 HIGH다. 켜진 뒤 낮은 입력 한 번으로 4가 되면 바로 꺼지므로 켜짐과 꺼짐의 지연이 대칭인 필터는 아니다.

22 사인파와 실제 속도



한 주기 / 청록: D9, 주황: D10 / 트림 0의 명령값

핵심 식

currentOffset=sin(phase)×65다. sin은 −1부터 +1 사이이고 위상은 한 바퀴마다 2π 증가한다. 좌우 순수 명령은 90+offset과 90−offset이다. 그림은 명령 궤적이며 실제 날개 센서 측정 결과가 아니다.

속도의 의미

스로틀 1050~2000을 반복문당 0.060~0.100 rad로 변환한다. 마지막 delay(4)만 가정하면 f≈Δphase/(2π×0.004)이므로 약 2.39~3.98 Hz다. 실제 반복문의 계산·시리얼 시간과 Servo 라이브러리의 펄스 갱신 주기, 서보 부하 때문에 이 값은 명령 위상의 근사 상한 추정이다.

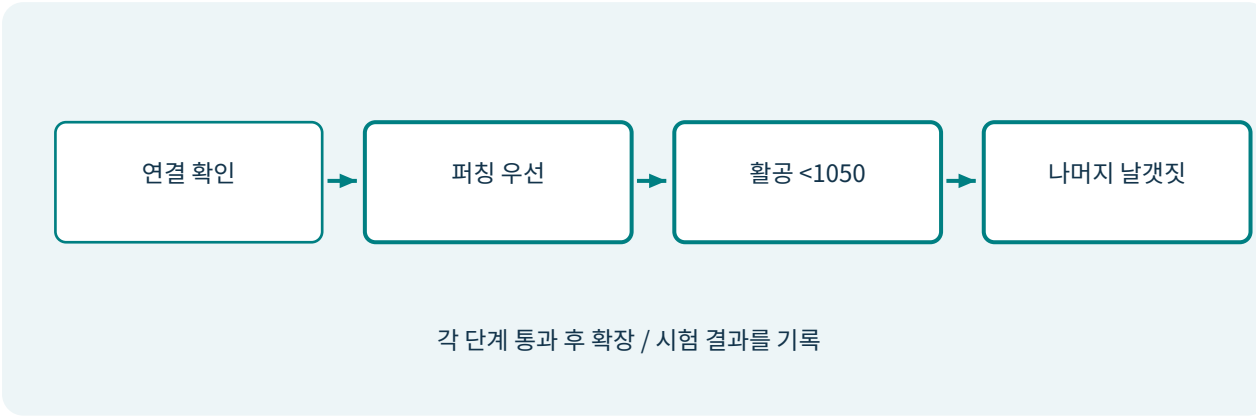
왜 중간이 빠른가

θ=90+A sin(ϕ)에서 시간당 위상 증가가 일정하면 각속도는 Aω cos(ϕ)다. 중심을 통과할 때 절댓값이 크고 끝에서 0이 된다. 양 끝의 급격한 방향 반전을 줄일 수 있지만, 이 파형만으로 에너지 절약이나 충분한 양력을 보장할 수는 없다.

개선 아이디어

millis 또는 micros의 실제 경과시간 dt로 phase+=2πf×dt를 계산하면 실행 속도 변화에 덜 의존한다. 이것은 원본과 다른 개선 설계다. dt 계산, 최대값 제한, 느려진 루프의 처리, 서보 추종성을 별도 시험한 뒤 적용한다.

23 활공·퍼칭·상태 유지



분기 우선순위

먼저 연결 상태를 검사한다. 연결 상태라면 퍼칭 상태를 갱신하고, 퍼칭이면 퍼칭 함수가 우선한다. 그렇지 않으면 throttle<1050에서 활공, 나머지는 날갯짓이다. 이름이 활공이라고 실제 공력 활공이 증명된 것은 아니다.

정확한 경계

throttle>1100이면 퍼칭 해제, 그렇지 않고 CH7>1600이며 throttle<1100이면 퍼칭 설정이다. CH7<1400에서도 해제한다. throttle=1100에서는 위 두 스로틀 조건에 걸리지 않아 이전 퍼칭 상태가 남을 수 있다. CH7=1400 또는 1600도 엄격 부등호에 주의한다.

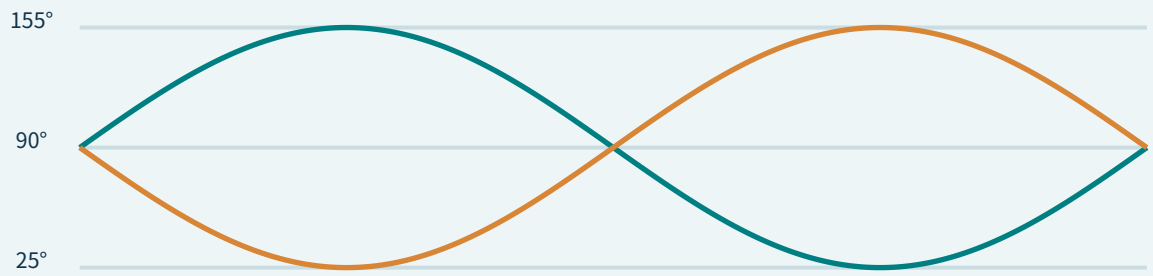
활공 복귀

활공 함수는 offset을 한 루프마다 2씩 0 쪽으로 이동하고 절댓값 2.5 미만이면 0으로 만든다. 조향·엘리베이터·트림은 뒤에서 더해지므로 두 서보가 반드시 90°로 멈추는 것은 아니다.

퍼칭 계산

CH6의 1000~2000을 maxUpAngle 30~60으로 map한다. goalAngle=maxUpAngle+20 sin(perchPhase)−20이므로 이상적 목표는 maxUpAngle−40부터 maxUpAngle까지다. CH6가 1000이면 −10~30, 2000이면 20~60이다. currentOffset은 1.5씩 목표를 따라가므로 실제 명령이 이 목표와 즉시 일치하지 않는다. CH6 피치 트림도 동시에 더해진다.

24 좌우 혼합·끝값 제한



한 주기 / 청록: D9, 주황: D10 / 트림 0의 명령값

최종 계산

outL=90+int(offset)+finalSteer+elevOffset+pitchTrim.
outR=90−int(offset)+finalSteer−elevOffset−pitchTrim. 조향은 같은 부호이고 나머지 두 자세 보정은 반대 부호다. 대칭 장착이라는 기계 조건과 함께 이해해야 한다.

끝값에 걸리는 사례

offset=65, finalSteer=30, elevOffset=15, pitchTrim=20이면 왼쪽 계산은 220°다. 마지막 함수에서 175°로 잘린다. 잘린 구간에서는 원래 사인파 모양이 유지되지 않으며 좌우 궤적이 달라질 수 있다. 큰 진폭과 큰 트림을 함께 쓰지 않도록 여유를 계산한다.

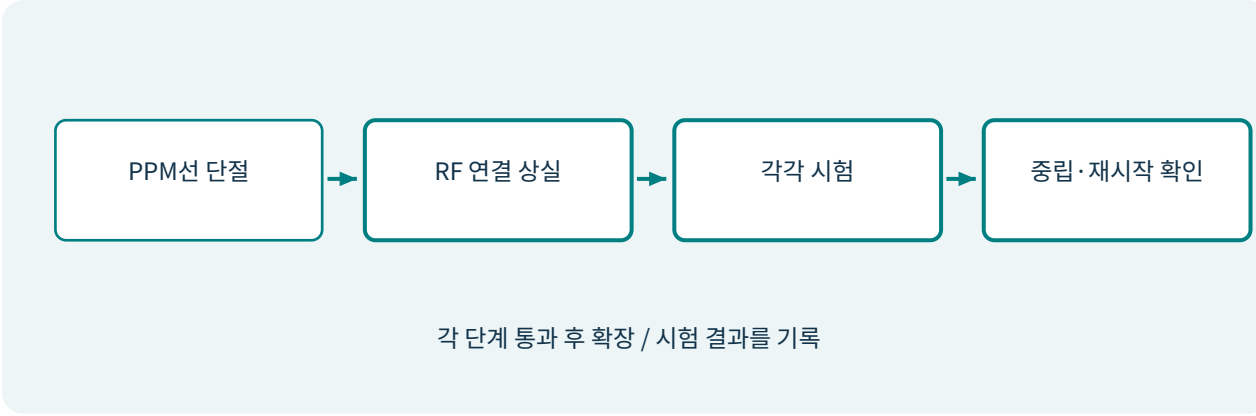
명령과 실제 위치

lastPosL과 lastPosR은 최근 전송한 명령을 보관한다. 실제 엔코더 피드백이 아니다.
abs(target−lastPos)>=1 조건은 정수 목표가 1° 이상 달라질 때 재전송한다. 기구가 걸려도 이 변수로는 감지할 수 없다.

개선 아이디어

좌우별 min/max와 중립, 방향 계수, 트림 범위를 분리하고 혼합 후 가능한 진폭을 제한하는 설계를 검토할 수 있다. 실제 기구 범위를 측정한 뒤 설정한다. 5~175라는 원본 제한은 특정 기구의 기계 안전 인증값이 아니다.

25 신호 상실과 재시작



원본의 판정

latestValidChannelValue(3,0)이 0인지로 연결을 판정한다. 유효값이 수신되었다는 사실과 방금 새 프레임이 도착했다는 사실은 다르다. 라이브러리가 과거 유효값을 보관하거나 수신기가 연결 상실 때도 정상 형식 PPM을 계속 출력하면 이 판정으로 놓칠 수 있다. [S3]

시간 감시만으로도 부족한 경우

새 PPM 프레임의 수신 시각을 감시하면 신호선 단절은 찾을 수 있다. 하지만 수신기가 RF 연결을 잃은 뒤에도 PPM을 계속 출력하면 프레임 시간 감시만으로는 RF 상실을 알 수 없다. 수신기의 failsafe 채널 또는 전용 링크 상태 정보를 함께 사용해야 한다.

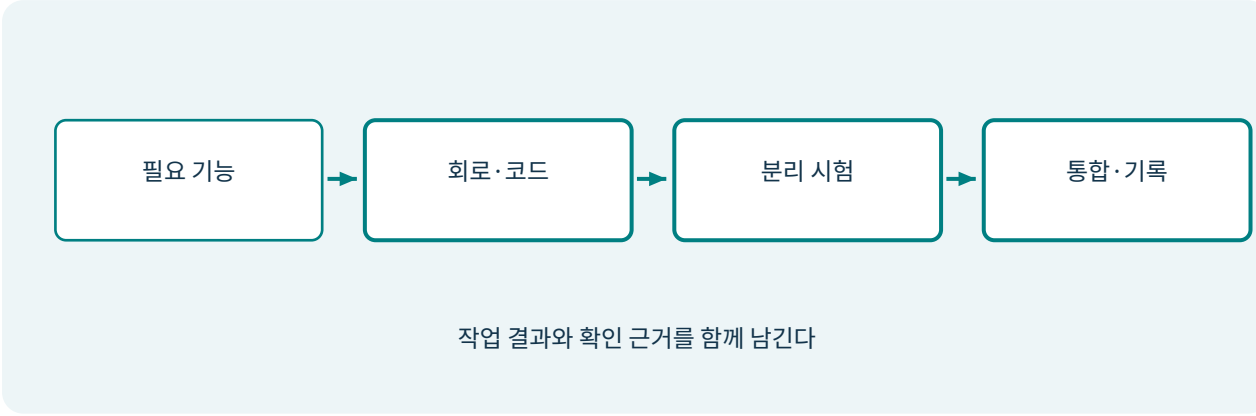
복귀와 재연결

processFailsafe는 offset을 1.5씩 중립으로 줄이고 LED를 끈다. 조향·피치 보정은 이 경로에서 적용하지 않는다. wasFailsafe=true를 설정한다. 재연결 후 날갯짓 함수에 들어오면 phase를 π로 재설정하지만 당시 offset과 바로 이어지지 않아 명령 불연속이 생길 수 있다.

시험과 개선

링크를 분리하고 조종기 OFF, 수신기 전원 OFF, PPM선 분리, 보드 재부팅을 각각 시험한다. 복구 후 낮은 스로틀 확인과 별도 재시작 허가가 필요한 설계를 권장한다. 원본에는 별도 ARM 채널이 없고 CH7은 퍼칭 채널이므로 ARM 스위치로 혼동하지 않는다.

26 코드에 추가할 기능과 주석 방식



주석의 원칙

무엇을 하는지뿐 아니라 왜 필요한지, 단위가 무엇인지, 어떤 전제가 있는지를 적는다. 예: flapAmplitude는 서보 명령의 중심 대비 각도이며 날개 실제 각도가 아니다. 코드의 동작과 다른 희망 기능을 주석만으로 구현됐다고 적지 않는다.

우선 검토 기능

실제 프레임 타임아웃, 수신기 failsafe 상태 판독, 시작 시 낮은 스로틀 확인, 별도 ARM 상태, 좌우 개별 기계 범위, 시간 기반 위상, 진폭 서서히 증가, 오류 상태 LED를 검토한다. 기존 채널과 핀을 재사용할 때 충돌을 먼저 조사한다.

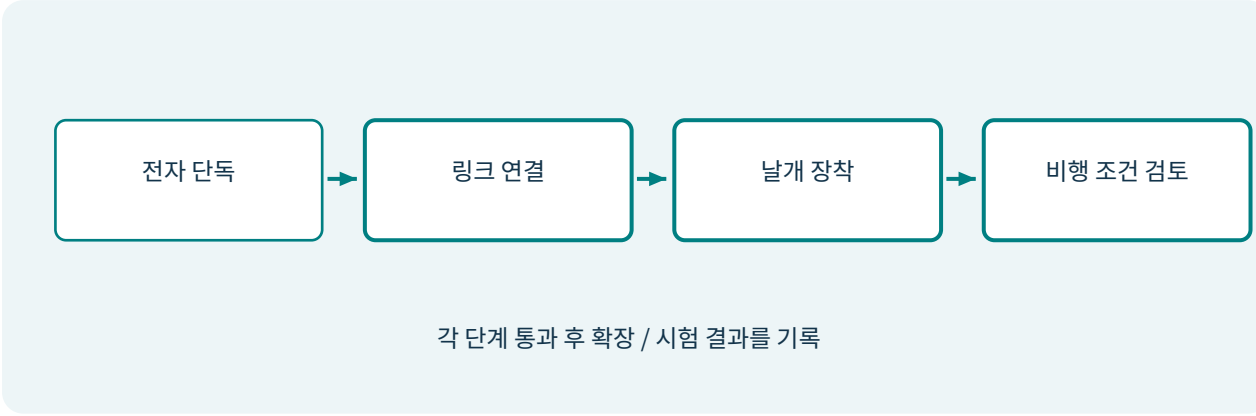
하드웨어가 필요한 기능

배터리 전압 감시는 ADC 입력용 저항 분압과 보정이 필요하다. 큰 LED는 트랜지스터나 MOSFET 구동부가 필요하다. 서보 과부하를 자동 감지하려면 전류 센서 등 추가 측정이 필요하다. 현재 배선과 코드에 이런 기능이 이미 있다고 쓰지 않는다.

검증 순서

기능 한 개 추가 → 컴파일 → 링크 없는 서보 시험 → 잘못된 입력 시험 → 전원 OFF/ON과 재연결 시험 → 변경 전후 기록 순서로 진행한다. 개선 예제는 원본 파일과 별도 이름으로 저장하고 변경 이유와 되돌리는 방법을 남긴다.

27 지상 단계별 시험



시험 1: 전자 단독

서보 혼과 날개를 분리한다. 채널과 모드가 예상대로 선택되는지 확인한다. 낮은 스로틀에서 시작하며 전압과 전류를 관찰한다. 양쪽 서보를 바로 큰 진폭으로 구동하지 않는다.

시험 2: 링크 연결

전원 차단 후 중립 기준으로 링크를 연결한다. 교육용 시작값으로 진폭 5~10° 같은 작은 범위를 검토하되 실제 기구에 맞춘다. 좌우 끝에서 간섭, 과도한 소음, 온도 상승, 전압 강하가 있는지 확인한다. 원본 65°는 첫 기계 시험의 필수값이 아니다.

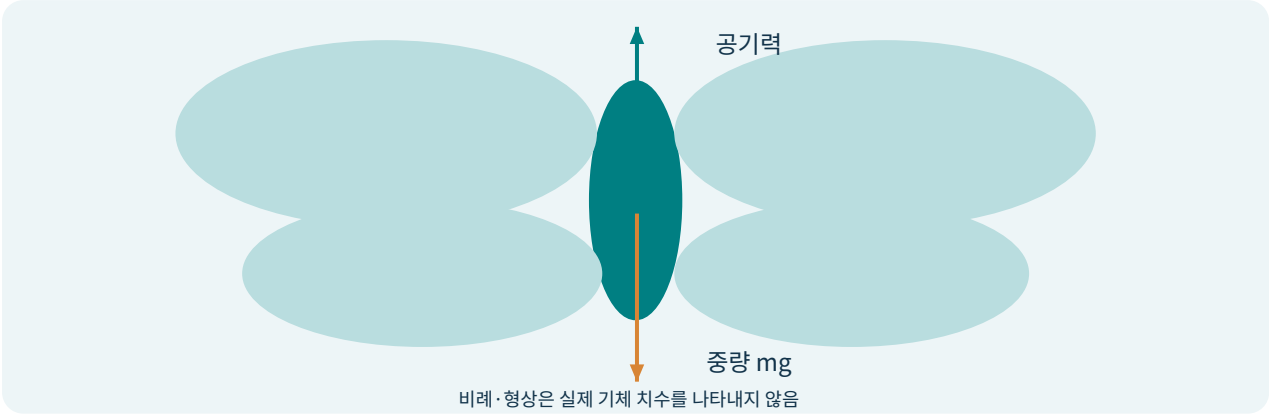
시험 3: 날개 장착

날개를 단 뒤 같은 주파수에서도 부하가 어떻게 늘어나는지 측정한다. 고속 영상으로 좌우 궤적과 서보 지연을 확인한다. 촬영 프레임 수와 시간으로 실제 주기를 계산한다. 반사 진동이나 고정부 움직임을 날개 운동으로 착각하지 않는다.

통과 기준

반복 구동 중 풀림·균열·걸림이 없어야 하고, 부하 전압과 전류 및 온도가 부품 정격 안에 있어야 한다. 조종기 OFF와 PPM선 분리에서 기대한 복귀가 확인돼야 한다. 기체의 구체적인 시험시간과 허용 온도는 해당 부품 자료를 보고 정한다.

28 비행을 이해하는 최소 원리



뜰 수 있으려면

자유비행에서 평균 수직 공기력이 중량 mg를 지탱하고 원하는 방향의 힘과 자세 안정성이 확보돼야 한다. 예를 들어 50 g이면 mg≈0.49 N이다. 이는 계산 예시로 실제 기체 질량이 아니다. 지상 고정 시험의 날갯짓만으로 양력이 충분하다고 단정하지 않는다.

무엇이 성능을 바꾸나

날개 면적과 형상, 막의 장력, 수동 비틀림, 실제 스트로크, 주파수, 좌우 차이, 총질량과 무게중심이 함께 영향을 준다. 부품이 더 강하다는 이유로 무거운 서보를 선택하면 전체 비행 조건이 나빠질 수 있다.

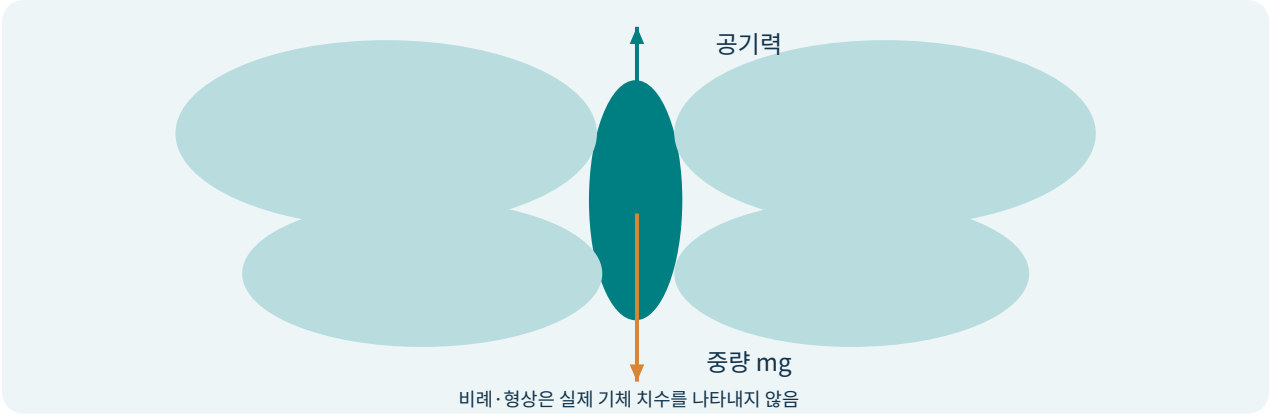
측정과 판단

반복 가능한 조건에서 정지 추력이나 수직 힘을 측정하는 장치를 설계할 수 있으나 고정구와 공기 흐름이 결과를 바꾼다. 측정값에 고정구 진동과 저울의 동적 응답이 섞이지 않는지 조사한다. 영상과 힘 측정을 구분하고 오차를 기록한다.

시험의 한계

본 자료에는 원본 기체 질량, 날개 형상·재료, 권장 무게중심, 실제 비행 로그가 없다. 따라서 비행 가능한 치수나 최종 설정은 확정할 수 없다. 이 장은 비행 조건을 확인하는 절차이며 실증 완료 보고서가 아니다.

29 첫 비행 준비와 기록



비례·형상은 실제 기체 치수를 나타내지 않음

사전 준비

주변 사람이 없는 통제 가능한 시험 공간에서 장치의 크기와 특성에 맞는 시험을 계획한다. 실내와 실외의 조건은 다르다. 현장 이용 조건과 적용되는 운영 규정은 시험 시점의 관할 정보를 확인한다. 본 책은 특정 기체의 법적 분류를 확정하지 않는다.

고정·제한 시험

먼저 날개 회전 범위 밖에서 전원과 무선 정지를 확인한다. 제한 시험에서 끈을 쓴다면 날개와 링크에 감기지 않게 하고 구속이 자세를 바꾼다는 점을 고려한다. 고정 시험을 통과했다는 이유만으로 자유비행을 자동 승인하지 않는다.

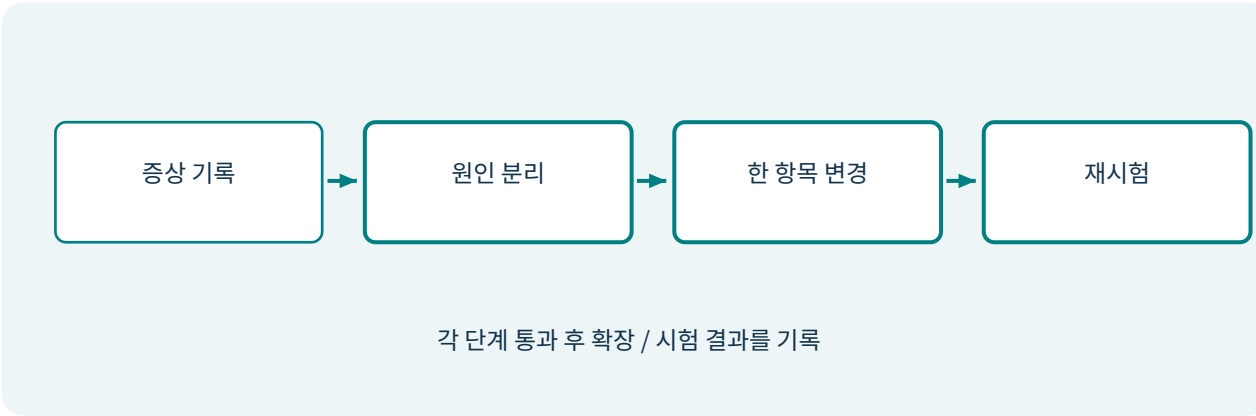
변수를 하나씩

좌우 방향·중립·질량·무게중심·진폭·주파수를 기록한다. 한 번에 하나의 변수만 바꾸고 전후를 비교한다. 위로 가지 못한다고 스로틀을 무조건 늘리지 말고 실제 스트로크 감소, 전압 강하, 기체 무게, 날개 형상부터 분리해서 판단한다.

종료와 점검

시험 뒤 전원을 차단하고 배터리, 커넥터, 혼 나사, 축, 출력물 균열을 조사한다. 충돌이나 과부하가 있었다면 전 회차와 같은 설정이라도 재검증한다. 결과는 성공/실패만 적지 말고 무엇이 관찰되었는지 수치와 영상 번호로 남긴다.

30 증상별 문제 해결



서보가 전혀 안 움직임

로직 시작 여부 → 실제 모터 전압 → GND 연결 → 신호 핀 → 최소 서보 예제 → 서보 교체 순으로 확인한다. PPM 통합 코드만 시험하면 모드 조건 때문에 멈춘 것과 배선 불량을 구분하기 어렵다.

떨림·재부팅·과열

떨림은 입력 불안정, 전원 강하, 접촉 불량, 기계 유격을 구분한다. 두 서보를 동시에 움직일 때만 재부팅하면 전원 경로를 우선 측정한다. 끝에서 웅거리면 즉시 작동을 멈추고 혼·링크 간섭과 펄스 범위를 확인한다.

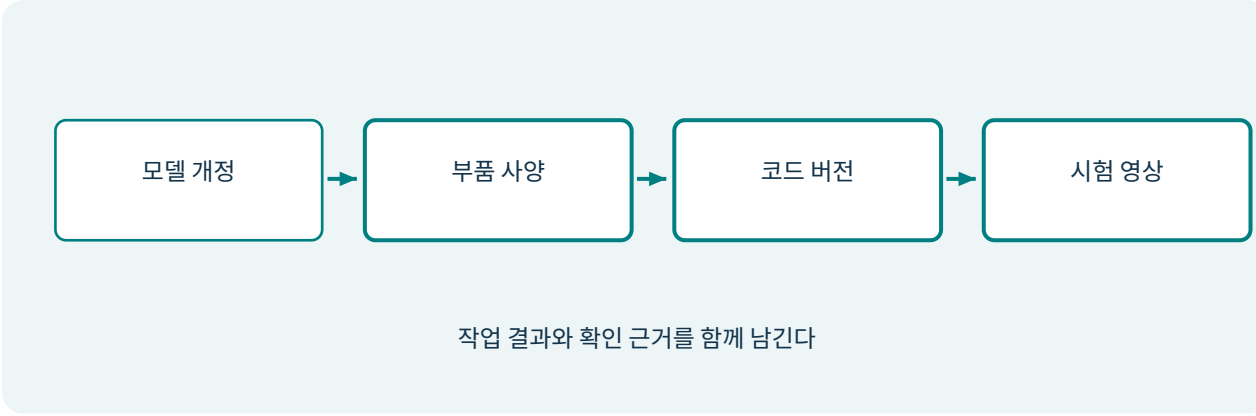
좌우가 다르게 움직임

D9와 D10의 실제 연결, 혼 장착 방향, 중립 오차, 링크 길이, 날개 질량 차이를 확인한다. 코드의 좌우 이름만 바꿔서 해결하려 하지 않는다. 출력 끝값에 한쪽만 잘리는지도 로그와 계산으로 조사한다.

모델·출력 문제

구멍이 막히면 CAD 윤곽 → Blender 면 생성 → STL → 슬라이서 층을 차례로 비교한다. 크기가 틀리면 최초 기준 길이와 exporter 단위 설정을 비교한다. 균열은 두께, 적층 방향, 체결 압력, 재료 상태를 분리해 점검한다.

31 정비·개정·공유



반복 제작 문서

출력 방향과 슬라이서 프로파일, 실제 측정 치수, 부품 모델, 코드 버전, 보드 패키지, 라이브러리 버전을 함께 남긴다. 사진에는 커넥터 방향과 혼 기준선을 표시한다. 수정한 파일 이름에 날짜와 개정 번호를 넣는다.

원본 저작자

제공 코드의 저작자 표기는 S-DiY channel / TruongVanSu91, 2026년 2월이다. 사용자가 제시한 조건은 개인 비상업적 취미 사용, 상업적 활용의 명시적 서면 동의, 소개 콘텐츠의 저자 표시 및 원본 링크 요구다. 이 책에 코드를 수록했다고 추가 사용 권한이 생기지 않는다.

출처와 공개 전 확인

원본 프로젝트 URL은 이 대화 자료에 확정되어 있지 않으므로 임의로 만들어 넣지 않았다. 책의 공개·판매·수익화 자료 활용을 계획한다면 원본 저자에게 그 범위를 확인하고 관련 권리를 검토한다. 원문 연락처는 Sdiychannel01@gmail.com이다. 다른 문서의 이용 허용 범위도 따로 확인한다.

남아 있는 실물 자료

완성 기체에 맞춘 확정판을 만들려면 실제 DXF와 .blend 또는 STL, 서보·보드·수신기·배터리 모델, 날개 치수와 질량, 조립 사진, 입력값 및 비행 시험 기록이 필요하다. 이 책의 작업 절차를 사용하면서 그 항목을 채워 제작 기록을 축적한다.

부록 A | 원본 코드 1/3

줄 번호는 복사 대상이 아닙니다. 다음 줄 들여쓰기는 인쇄용 줄바꿈입니다. 실제 업로드 전 보드·라이브러리·서보 사양을 확인하세요.

```
001  /*
002   * S-DiY RC Bionic Butterfly project
003   * PROJECT: S-DiY 2 SERVO FLAP RC BIONIC BUTTERFLY
004   * VERSION: 1.0 (SINE WAVE)
005   * AUTHOR:  [S-DiY channel/ TruongVanSu91]
006   * DATE:    02/2026
007   * COPYRIGHT (C) 2026. ALL RIGHTS RESERVED.
008   * ---------------------------------------------------------------------------
009   */
010  #include <Servo.h>
011  #include <PPMReader.h>
012  const byte interruptPin = 3;
013  const byte ledPin = 6;
014  const byte channelAmount = 8;
015  PPMReader ppm(interruptPin, channelAmount);
016  Servo servoLeft, servoRight;
017  const int centerAngle = 90;
018  const int flapAmplitude = 65;
019  const int maxPitchTrim = 20;
020  float phase = 0, perchPhase = 0, currentOffset = 0;
021  bool isPerchingState = false, wasFailsafe = true;
022  int lastPosL = 90, lastPosR = 90;
023  int lastChValues[8] = {1500, 1500, 1500, 1500, 1500, 1500, 1500, 1500};
024  const int ppmDeadband = 3;
025  int ledDebounceCounter = 0;
026  const int ledThreshold = 5;
027  unsigned long lastLogTime = 0;
028  const int logInterval = 1000;
029  void setup() {
030    Serial.begin(115200);
031    pinMode(ledPin, OUTPUT);
032    digitalWrite(ledPin, LOW);
033    servoLeft.attach(9, 500, 2500);
034    servoRight.attach(10, 500, 2500);
035    servoLeft.write(centerAngle);
036    servoRight.write(centerAngle);
037    Serial.println("--- BIONIC BUTTERFLY READY (SINE WAVE MODE) ---");
038  }
039  int readPPMFiltered(byte ch) {
040    int raw = ppm.latestValidChannelValue(ch + 1, 1500);
041    if (abs(raw - lastChValues[ch]) > ppmDeadband) {
042      lastChValues[ch] = raw;
043    }
044    return lastChValues[ch];
045  }
046  void loop() {
047    unsigned long ch3Raw = ppm.latestValidChannelValue(3, 0);
048    bool isConnected = (ch3Raw != 0);
049    if (!isConnected) {
```

부록 A | 원본 코드 2/3

줄 번호는 복사 대상이 아닙니다. 다음 줄 들여쓰기는 인쇄용 줄바꿈입니다. 실제 업로드 전 보드·라이브러리·서보 사양을 확인하세요.

```
050      digitalWrite(ledPin, LOW);
051      processFailsafe();
052    } else {
053      int steering = readPPMFiltered(0);
054      int elevator = readPPMFiltered(1);
055      int throttle = readPPMFiltered(2);
056      int ch5Trim   = readPPMFiltered(4);
057      int rawCh6    = readPPMFiltered(5);
058      int ch7Switch = readPPMFiltered(6);
059      int ch8Raw    = readPPMFiltered(7);
060      if (ch8Raw > 1550) { if (ledDebounceCounter < ledThreshold) ledDebounceCounter++; }
061      else if (ch8Raw < 1450) { if (ledDebounceCounter > 0) ledDebounceCounter--; }
062      digitalWrite(ledPin, (ledDebounceCounter >= ledThreshold) ? HIGH : LOW);
063      int steerValue = (abs(steering - 1500) > 10) ? map(steering, 900, 2100, -15, 15) : 0;
064      int finalSteer = steerValue + map(ch5Trim, 1000, 2000, -15, 15);
065      int elevOffset = (abs(elevator - 1500) > 10) ? map(elevator, 900, 2100, -15, 15) : 0;
066      int pitchTrim = map(rawCh6, 1000, 2000, -maxPitchTrim, maxPitchTrim);
067      if (throttle > 1100) isPerchingState = false;
068      else if (ch7Switch > 1600 && throttle < 1100) isPerchingState = true;
069      if (ch7Switch < 1400) isPerchingState = false;
070      if (isPerchingState) {
071        updateCurrentOffsetPerch(rawCh6);
072      } else if (throttle < 1050) {
073        updateCurrentOffsetGlide();
074      } else {
075        updateCurrentOffsetSine(throttle);
076      }
077      int outL = centerAngle + (int)currentOffset + finalSteer + elevOffset + pitchTrim;
078      int outR = centerAngle - (int)currentOffset + finalSteer - elevOffset - pitchTrim;
079      updateServos(outL, outR);
080    }
081    if (millis() - lastLogTime > logInterval) {
082      lastLogTime = millis();
083      if (isConnected) {
084        Serial.print("[CONNECTED] CH3: "); Serial.print(lastChValues[2]);
085        Serial.print(" | LED: "); Serial.println(digitalRead(ledPin) ? "ON" : "OFF");
086      } else {
087        Serial.println("[!] WARNING: NO SIGNAL DETECTED");
088      }
089    }
090    delay(4);
091  }
092  void updateCurrentOffsetSine(int throttle) {
093    if (wasFailsafe) { phase = PI; wasFailsafe = false; }
094    float speedFactor = map(throttle, 1050, 2000, 60, 100) / 1000.0;
095    phase += speedFactor;
096    if (phase >= 2 * PI) phase -= 2 * PI;
097    currentOffset = sin(phase) * flapAmplitude;
098  }
```

부록 A | 원본 코드 3/3

줄 번호는 복사 대상이 아닙니다. 다음 줄 들여쓰기는 인쇄용 줄바꿈입니다. 실제 업로드 전 보드·라이브러리·서보 사양을 확인하세요.

```
099  void updateCurrentOffsetPerch(int rawCh6) {
100    wasFailsafe = true;
101    float maxUpAngle = map(rawCh6, 1000, 2000, 30, 60);
102    perchPhase += 0.01;
103    float goalAngle = maxUpAngle + (sin(perchPhase) * 20.0) - 20.0;
104    if (abs(currentOffset - goalAngle) > 0.5) currentOffset += (currentOffset < goalAngle) ?
     1.5 : -1.5;
105  }
106  void updateCurrentOffsetGlide() {
107    wasFailsafe = true;
108    if (currentOffset < 0) currentOffset += 2.0;
109    else if (currentOffset > 0) currentOffset -= 2.0;
110    if (abs(currentOffset) < 2.5) currentOffset = 0;
111  }
112  void processFailsafe() {
113    if (currentOffset < 0) currentOffset += 1.5;
114    else if (currentOffset > 0) currentOffset -= 1.5;
115    if (abs(currentOffset) < 1.5) currentOffset = 0;
116    updateServos(centerAngle + (int)currentOffset, centerAngle - (int)currentOffset);
117    wasFailsafe = true;
118  }
119  void updateServos(int targetL, int targetR) {
120    targetL = constrain(targetL, 5, 175);
121    targetR = constrain(targetR, 5, 175);
122    if (abs(targetL - lastPosL) >= 1) { servoLeft.write(targetL); lastPosL = targetL; }
123    if (abs(targetR - lastPosR) >= 1) { servoRight.write(targetR); lastPosR = targetR; }
124  }
```

부록 B | 작은 왕복 운동 교육 코드

원본과의 차이

PPM을 읽지 않고 작은 펄스 범위만 시험하는 별도 교육 예제다. 1000~2000 µs와 1500 µs 중립 역시 실제 서보 사양을 확인해야 한다. 명령 범위를 작게 했다고 모든 기구에서 안전이 보장되는 것은 아니다. 이 환경에서는 컴파일하거나 실물 구동하지 않았다.

```
// 독립 교육 예제: RC 수신 없이 디지털 서보 두 개의 작은 왕복 시험
// 비행 제어 코드가 아닙니다. 날개와 링크를 분리한 상태에서 시작합니다.
#include <Servo.h>
Servo leftServo, rightServo;
const int LEFT_PIN = 9, RIGHT_PIN = 10;
const int CENTER_US = 1500; // 실제 서보 중립 펄스로 확인 후 수정
const int SWING_US = 50; // 중심에서 한쪽으로 50us인 작은 시험값
void setup() {
leftServo.attach(LEFT_PIN, 1000, 2000);
rightServo.attach(RIGHT_PIN, 1000, 2000);
leftServo.writeMicroseconds(CENTER_US);
rightServo.writeMicroseconds(CENTER_US);
delay(2000); // 시작 자세를 확인할 시간
}
void loop() {
// 0.5Hz: 명령 궤적 한 주기 2초. 시간 경과로 위상을 계산합니다.
float phase = (millis() % 2000UL) * (2.0 * PI / 2000.0);
int delta = (int)(SWING_US * sin(phase));
leftServo.writeMicroseconds(CENTER_US + delta);
rightServo.writeMicroseconds(CENTER_US - delta);
delay(20); // 시험용 갱신 간격
}
```

부록 C | 코드 주석 해설 1

10~16행 | 라이브러리와 하드웨어 객체

Servo는 목표 위치 펄스를, PPMReader는 입력 채널을 다룬다. interruptPin=3은 수신기 신호이고 ledPin=6은 LED다. 8채널을 기대한다. ppm과 servoLeft/servoRight는 이후 함수가 공통으로 사용하는 객체다.

// D3는 PPM 입력. 사용한 보드에서 인터럽트 지원 확인

// D9/D10은 좌우 서보 신호. 좌우는 실제 장착으로 확인

17~28행 | 설정과 기억 상태

centerAngle은 90, flapAmplitude는 65, maxPitchTrim은 20이다. phase는 날갯짓 위상, perchPhase는 퍼칭 위상, currentOffset은 현재 운동 오프셋이다. wasFailsafe는 이름과 달리 활공·퍼칭을 거친 경우에도 참이다. lastPos는 실제 위치 센서값이 아니다.

// 진폭 65는 중심 대비 ±65도의 명령 범위

// lastPos는 직전 명령 기억. 실제 각도를 읽은 값 아님

위 줄 번호는 이 책에 수록한 원문 기준입니다. 소스에 줄을 추가하면 번호가 달라집니다. 주석은 검토 이유이며 기능 구현 자체가 아닙니다.

부록 C | 코드 주석 해설 2

29~45행 | 시작과 입력 필터

시리얼 속도를 설정하고 LED를 끄며 서보를 연결한다. 500/2500은 write 각도 좌표를 펄스로 변환하는 양 끝 기준이다. 디지털 서보라 해도 이 범위를 모두 지원하지는 않는다. 필터는 입력 변화량이 3보다 클 때만 기억을 갱신한다. 기본값 1500은 신호가 유효함을 의미하지 않는다.

// 입력 ch는 0부터. 라이브러리 호출은 ch+1

// 이 필터는 평균 필터가 아니라 변화 문턱 필터

46~62행 | 연결과 LED

CH3에서 유효값 0 여부를 본다. 반환값의 신선도를 별도로 확인하지 않는다. 수신이 끊기면 LED를 끄고 복귀 함수를 부른다. 단 LED 카운터 자체는 이 경로에서 0으로 초기화하지 않아 재연결 때 과거 상태가 남을 수 있다.

// 연결 판정은 라이브러리와 수신기 동작에 의존

// LED 카운터의 신호 상실 시 초기화 필요성 검토

위 줄 번호는 이 책에 수록한 원문 기준입니다. 소스에 줄을 추가하면 번호가 달라집니다. 주석은 검토 이유이며 기능 구현 자체가 아닙니다.

부록 C | 코드 주석 해설 3

63~79행 | 자세 혼합과 모드 선택

조향·엘리베이터에 작은 중심 무시 구간을 적용하고 트림을 더한다. 퍼칭은 스로틀·CH7 및 이전 상태로 결정한다. 조건이 참이면 퍼칭이 활공보다 우선한다. 계산한 좌우 목표는 updateServos로 전달된다.

// CH6는 피치 트림과 퍼칭 범위에 동시에 사용

// 마지막 constrain에 걸리는지 혼합 최대값 계산

81~98행 | 로그와 사인파

로그는 약 1초마다 출력하며 측정 위치가 아니라 채널과 LED 상태를 보여준다. delay(4)는 루프 지연이고 전체 루프 시간이 정확히 4ms라는 뜻은 아니다. 날갯짓 진입 시 위상을 π로 리셋하고 Δphase를 더한 다음 sin을 계산한다.

// speedFactor 단위: rad/loop. Hz나 RPM이 아님

// 명령 위상과 실제 날개 주파수는 따로 측정

위 줄 번호는 이 책에 수록한 원문 기준입니다. 소스에 줄을 추가하면 번호가 달라집니다. 주석은 검토 이유이며 기능 구현 자체가 아닙니다.

부록 C | 코드 주석 해설 4

99~111행 | 퍼칭과 활공

퍼칭 목표는 느린 사인파이며 목표와 0.5 넘게 차이 날 때 offset을 1.5씩 바꾼다. 남은 오차보다 큰 보폭 때문에 목표 근처에서 작은 왕복이 생길 수 있다. 활공은 2씩 중립 쪽으로 가고 충분히 가까워지면 0으로 만든다.

// 퍼칭은 착지 감지가 아니라 조종기 선택 연출

// 시간 기반 보간과 목표 초과 방지 개선을 검토

112~124행 | 신호 상실 복귀와 출력

신호 상실 경로는 offset을 0으로 줄이며 조향과 피치 보정을 빼고 양쪽 중심으로 돌아간다. 최종 출력은 5~175로 제한한다. 최근 명령과 1도 이상 차이가 나면 새 목표를 보낸다. 이 함수는 전류·충돌·실제 위치를 측정하지 않는다.

// 5~175는 원본 소프트웨어 제한값

// 실물 링크에서 측정한 좌우 개별 한계가 아님

위 줄 번호는 이 책에 수록한 원문 기준입니다. 소스에 줄을 추가하면 번호가 달라집니다. 주석은 검토 이유이며 기능 구현 자체가 아닙니다.

부록 D-1 | 부품 사양 기록

값을 확인한 뒤 작성합니다. 빈칸은 미확인 상태이며 임의 수치로 채우지 않습니다.

항목	측정값 / 확인 근거
서보 왼쪽/오른쪽 모델	_____
서보 전압 / 펄스 / 갱신율	_____
서보 질량 / 토크 / 속도	_____
보드 모델 / MCU / 전압 / 클록	_____
RAW 전원 회로 확인 자료	_____
수신기 모델 / PPM 포트	_____
수신기 신호레벨 / failsafe 출력	_____
배터리 셀 수 / 최대전압 / 용량	_____
레귤레이터 출력 / 정격 / 순간전류	_____
날개 치수 / 재료 / 기체 총질량	_____

부록 D-2 | 치수와 출력 기록

값을 확인한 뒤 작성합니다. 빈칸은 미확인 상태이며 임의 수치로 채우지 않습니다.

항목	측정값 / 확인 근거
원본 DXF 이름 / 기준 길이	_____
Blender 버전 / 단위 / 크기	_____
구멍 중심 / 지름 / 두께	_____
STL exporter Scale / Scene Unit	_____
슬라이서 치수 / 출력 방향	_____
프린터 / 노즐 / 재료	_____
층높이 / 외벽 / 인필	_____
서포트 / 첫 층 상태	_____
출력 치수 / 질량 / 조립 여유	_____
개정 번호 / 비교 사진	_____

부록 D-3 | 채널·정지 시험 기록

값을 확인한 뒤 작성합니다. 빈칸은 미확인 상태이며 임의 수치로 채우지 않습니다.

항목	측정값 / 확인 근거
CH1 최소 / 중앙 / 최대 / 기능	_____
CH2 최소 / 중앙 / 최대 / 기능	_____
CH3 최소 / 최대 / 낮은 스로틀값	_____
CH5 / CH6 실제 범위	_____
CH7 스위치 3단계 값	_____
CH8 스위치값 / LED 결과	_____
조종기 OFF 시 채널 / 동작	_____
PPM선 분리 시 채널 / 동작	_____
수신기 전원 OFF 시 동작	_____
재연결 / 보드 재시작 시 동작	_____

부록 D-4 | 지상·비행 시험 기록

값을 확인한 뒤 작성합니다. 빈칸은 미확인 상태이며 임의 수치로 채우지 않습니다.

항목	측정값 / 확인 근거
날짜 / 장소 / 시험자	_____
코드 / 도면 / 출력 개정	_____
기체 질량 / 무게중심기준	_____
날개 / 링크 / 중립 / 한계	_____
목표 및 실제 주파수	_____
실제 스트로크 / 영상 프레임률	_____
동작 중 전압 / 전류 / 온도	_____
모드 전환 / 수신 상실 결과	_____
관찰 결과 / 중단 원인	_____
다음 회차 변경 항목 한 개	_____

부록 E | 빠른 용어 사전

DXF

CAD 교환 형식. 선과 문자 등이 섞일 수 있다.

Mesh

정점·변·면으로 표현한 형상.

Normal

면이 향하는 방향. 닫힌 출력 부피의 안팎을 조사할 때 사용.

Non-manifold

면 연결이 일반적인 닫힌 부피 규칙과 맞지 않는 상태.

Solidify

표면에 두께를 만드는 Blender Modifier.

Boolean Difference

절삭체 부피를 다른 몸체에서 빼는 연산.

STL

삼각형 표면 형식. 물리 단위를 확실히 보존하는 형식이 아니다.

PPM

여러 RC 채널을 시간 간격으로 나타내는 신호.

Servo pulse

서보에 목표 위치를 전달하는 펄스. 모터 공급 전압과 구분.

Amplitude

중심에서 한쪽 끝까지의 크기.

Phase

반복 운동에서 진행된 위치.

Trim

중심 자세나 조향을 조금 이동시키는 보정.

Failsafe

신호 상실 등에서 정해진 상태로 옮기는 처리.

ARM

구동을 별도로 허가한 상태. 원본에는 독립 ARM 상태가 없다.

부록 F | 참고 자료와 출처

S1 | SparkFun, Pro Micro & Fio V3 Hookup Guide

기존 Pro Micro의 RAW/VCC 설명. USB-C 복제 보드의 사양을 대신하지 않는다.

https://learn.sparkfun.com/tutorials/pro-micro--fio-v3-hookup-guide/hardware-overview-pro-micro

S2 | Arduino Servo library API

attach, write, writeMicroseconds, read의 의미. 실제 부품의 허용 범위는 서보 자료로 확인한다.

https://github.com/arduino-libraries/Servo/blob/master/docs/api.md

S3 | Nikkilae, PPM-reader

인터럽트 기반 PPM 읽기, 1부터 시작하는 채널 번호, 최신 유효값 API. 사용자가 설치한 버전은 미확인이다.

https://github.com/Nikkilae/PPM-reader

S4 | Arduino, Troubleshoot servo motors

서보 신호·전원 점검 참고.

https://support.arduino.cc/hc/en-us/articles/360017053760-Troubleshoot-servo-motors

S5 | Blender Manual, STL

Selection Only, Batch, Scale, Scene Unit, Apply Modifiers 설명. 본문 4.0 기준 화면과 옵션 차이가 있을 수 있다.

https://docs.blender.org/manual/en/5.1/files/import_export/stl.html

S6 | SparkFun, Qwiic Pro Micro USB-C guide

기존 Pro Micro와 다른 전원 구성이 있는 공식 사례. 사용자가 가진 보드를 이 모델로 확정하지 않는다.

https://learn.sparkfun.com/tutorials/qwiic-pro-micro-usb-c-atmega32u4-hookup-guide

웹 자료 확인일: 2026-11-11. 일부 Blender 4.0 세부 문서와 PPMReader 소스 본문은 검색 환경에서 직접 확인되지 않아 버전별 단정은 피했다. 기계·출력·시험 절차는 본 교재의 교육용 설명이며 해당 원본 기체의 실측 결과를 뜻하지 않는다.

부록 G | 자료별 검증 상태

사용자 코드

제공된 전체 텍스트를 읽고 입력·모드·출력 계산을 정적으로 검토했다. 8채널 배정과 핀 및 기본값은 이 코드에서 확인했다. 실제 컴파일과 모터 동작 검증은 미실시다.

사용자 이미지

wirring.jpg의 배선과 라벨을 직접 확인했다. 파일 관리자 캡처는 도면 파일이 존재한다는 화면 정보만 제공한다. 캡처를 도면의 치수 근거로 사용하지 않았다.

교재 그림

각 장의 그림은 이 책을 위해 그린 원리 설명용 벡터 그림이다. 실제 CAD 형상, 공차, 회로 보호부품, 승인된 PCB 배선을 나타내지 않는다. 사인파 그래프는 원본 명령식으로 계산한 그림이다.

실제 완성 단계

부품 모델 확정 → 치수 복구 검증 → 출력물 실측 → 전원 정격 검사 → 하드웨어 컴파일·업로드 → 신호 상실 시험 → 지상 성능 측정 → 비행 조건 판단이 남아 있다. 이 책은 그 과정을 수행하고 기록하는 기반 자료다.